

sync

the correctness problem

two threads?

introduces *non-determinism*

which one runs first?

allows for “race condition” bugs

... to be avoided with synchronization constructs

a threaded server?

```
Deposit(accountNumber, amount) {  
    account = GetAccount(accountId);  
    account->balance += amount;  
    SaveAccountUpdates(account);  
}
```

maybe GetAccount/SaveAccountUpdates can be slow?

read/write disk sometimes? contact another server sometimes?

maybe lots of requests to process?

maybe real logic has more checks than Deposit()

...

all reasons to handle multiple requests at once

→ many threads all running the server loop

the lost write

`account->balance += amount;` (in two threads; same account)

Thread A

```
mov account->balance, %rax  
add amount, %rax
```

context switch

Thread B

```
mov account->balance, %rax  
add amount, %rax
```

context switch

```
mov %rax, account->balance
```

context switch

```
mov %rax, account->balance
```

the lost write

`account->balance += amount;` (in two threads; same account)

Thread A

```
mov account->balance, %rax  
add amount, %rax
```

context switch

```
mov %rax, account->balance
```

“lost write”

context switch

Thread B

```
mov account->balance, %rax  
add amount, %rax
```

```
mov %rax, account->balance
```

“winner” of race

thinking about race conditions (1)

what are the possible values of x ? (initially $x = y = 0$)

Thread A

$x \leftarrow 1$

Thread B

$y \leftarrow 2$

thinking about race conditions (1)

what are the possible values of x ? (initially $x = y = 0$)

Thread A

Thread B

$x \leftarrow 1$

$y \leftarrow 2$

must be 1. Thread B can't do anything

thinking about race conditions (2)

possible values of x ? (initially $x = y = 0$)

Thread A

$$x \leftarrow y + 1$$

Thread B

$$y \leftarrow 2$$

$$y \leftarrow y \times 2$$

thinking about race conditions (2)

possible values of x ? (initially $x = y = 0$)

Thread A

$x \leftarrow y + 1$

Thread B

$y \leftarrow 2$

$y \leftarrow y \times 2$

if A goes first, then B: 1

if B goes first, then A: 5

if B line one, then A, then B line two: 3

thinking about race conditions (3)

what are the possible values of x ?

(initially $x = y = 0$)

Thread A

$x \leftarrow 1$

Thread B

$x \leftarrow 2$

thinking about race conditions (3)

what are the possible values of x ?

(initially $x = y = 0$)

Thread A

$x \leftarrow 1$

Thread B

$x \leftarrow 2$

1 or 2

thinking about race conditions (3)

what are the possible values of x ?

(initially $x = y = 0$)

<i>Thread A</i>	<i>Thread B</i>
$x \leftarrow 1$	$x \leftarrow 2$

1 or 2

... but why not 3?

B: x bit 0 $\leftarrow$ 0

A: x bit 0 $\leftarrow$ 1

A: x bit 1 $\leftarrow$ 0

B: x bit 1 $\leftarrow$ 1

thinking about race conditions (2, reprise)

possible values of x ? (initially $x = y = 0$)

Thread A

$x \leftarrow y + 1$

Thread B

$y \leftarrow 2$

$y \leftarrow y \times 2$

why not 7?

B (start): $y \leftarrow 2 = 0010_{\text{TWO}}$; then $y \text{ bit } 3 \leftarrow 0$; $y \text{ bit } 2 \leftarrow 1$; then

A: $x \leftarrow 110_{\text{TWO}} + 1 = 7$; then

B (finish): $y \text{ bit } 1 \leftarrow 0$; $y \text{ bit } 0 \leftarrow 0$

atomic operation

atomic operation = operation that runs to completion or not at all

we will use these to let threads work together

most machines: loading/storing (aligned) words is atomic

so can't get 3 from $x \leftarrow 1$ and $x \leftarrow 2$ running in parallel

aligned $\approx$ address of word is multiple of word size (typically done by compilers)

but some instructions are not atomic; examples:

x86: integer add constant to memory location

many CPUs: loading/storing values that cross cache blocks

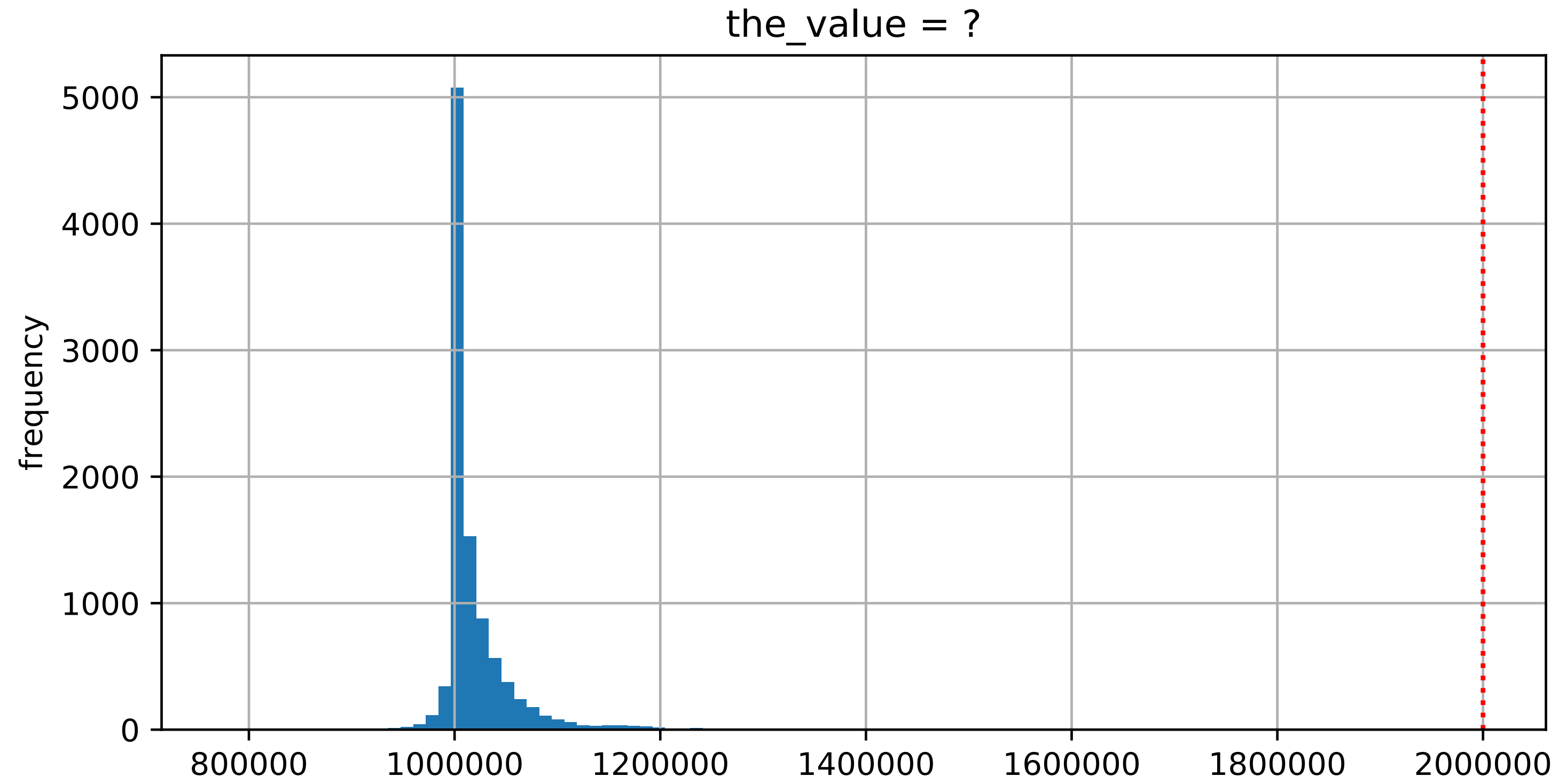
e.g. if cache blocks 0x40 bytes, load/store 4 byte from addr. 0x3E is not atomic

lost adds (program)

```
.global update_loop
update_loop:
    addl $1, the_value // the_value (global variable) += 1
    dec %rdi           // argument 1 -= 1
    jg update_loop     // if argument 1 >= 0 repeat
    ret
```

```
int the_value;
extern void *update_loop(void *);
int main(void) {
    the_value = 0;
    pthread_t A, B;
    pthread_create(&A, NULL, update_loop, (void*) 1000000);
    pthread_create(&B, NULL, update_loop, (void*) 1000000);
    pthread_join(A, NULL); pthread_join(B, NULL);
    // expected result: 1000000 + 1000000 = 2000000
    printf("the_value = %d\n", the_value);
}
```

lost adds (results)



but how?

probably not possible on single core

- exceptions can't occur in the middle of add instruction

... but 'add to memory' implemented with multiple steps

- still needs to load, add, store internally

- can be interleaved with what other cores do

(and actually it's more complicated than that — we'll talk later)

so, what is actually atomic

for now we'll assume: load/stores of 'words'
(64-bit machine = 64-bits words)

in general: *processor designer will tell you*

their job to design caches, etc. to work as documented

compilers move loads/stores (1)

```
void WaitForReady() {  
    do {} while (!ready);  
}
```

WaitForReady:

```
    movl ready, %eax           // eax ← other_ready  
  
    // value only loaded before loop  
.L2:  
    testl %eax, %eax  
    je .L2                    // while (eax == 0) repeat  
    ...
```

C standard says: can assume no other thread
So, it's fine to load ready in advance and reuse that

compilers move loads/stores (2)

```
void WaitForReady() {  
    is_waiting = 1;  
    do {} while (!ready);  
    is_waiting = 0;  
}
```

```
WaitForOther:  
    movl ready, %eax  
.L2:  
    testl %eax, %eax  
    je .L2           // while (eax == 0) repeat  
    movl $0, is_waiting  
    ret
```

C standard says: can assume no other thread

So, can do assignments `is_waiting` anytime before return

And `is_waiting = 1; is_waiting = 0` same as `is_waiting = 0`

fixing compiler reordering?

isn't there a way to tell compiler not to do these optimizations?

yes, but that is *still not enough*!

processors sometimes do this kind of reordering too
(between cores)

pthread and reordering

many pthreads functions *prevent reordering*

everything before function call actually happens before

includes *preventing some optimizations*

e.g. keeping global variable in register for too long

pthread_create, pthread_join, other tools we'll talk about ...

basically: if pthreads is waiting for/starting something, no weird ordering

implementation part 1: prevent compiler reordering

implementation part 2: use special instructions

example: x86 mfence instruction

some definitions

mutual exclusion: ensuring only one thread does a particular thing at a time
like updating shared balance

critical section: code that exactly one thread can execute at a time
result of mutual exclusion

lock: object only one thread can hold at a time
interface for creating critical sections

lock analogy

agreement: only change account balances while wearing this hat

normally hat kept on table

put on hat when editing balance

hopefully, only one person (= thread) can wear hat a time

need to wait for them to remove hat to put it on

“lock (or acquire) the lock” = get and put on hat

“unlock (or release) the lock” = put hat back on table

the lock primitive

locks: an object with (at least) two operations:

acquire or *lock* — wait until lock is free, then “grab” it

release or *unlock* — let others use lock, wakeup waiters

typical usage: everyone acquires lock before using shared resource

forget to acquire lock? weird things happen

```
Lock(account_lock);  
balance += ...;  
Unlock(account_lock);
```

the lock primitive

locks: an object with (at least) two operations:

acquire or *lock* — *wait* until lock is free, then “grab” it

release or *unlock* — let others use lock, wakeup waiters

typical usage: everyone acquires lock before using shared resource

forget to acquire lock? weird things happen

```
Lock(account_lock);  
balance += ...;  
Unlock(account_lock);
```

waiting for lock?

when waiting — ideally:

not using processor (at least if waiting a while)

OS can context switch to other programs

pthread mutex

```
#include <pthread.h>

pthread_mutex_t account_lock;
pthread_mutex_init(&account_lock, NULL);
    // or: pthread_mutex_t account_lock =
    //      PTHREAD_MUTEX_INITIALIZER;
...
pthread_mutex_lock(&account_lock);
balance += ...;
pthread_mutex_unlock(&account_lock);
...
pthread_mutex_destroy(&account_lock);
```

exercise

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
string one = "init one", two = "init two";
void ThreadA() {
    pthread_mutex_lock(&lock1);
    one = "one in ThreadA"; // (A1)
    pthread_mutex_unlock(&lock1);
    pthread_mutex_lock(&lock2);
    two = "two in ThreadA"; // (A2)
    pthread_mutex_unlock(&lock2);
}
void ThreadB() {
    pthread_mutex_lock(&lock1);
    one = "one in ThreadB"; // (B1)
    pthread_mutex_lock(&lock2);
    two = "two in ThreadB"; // (B2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}
```

possible values of one/two after A+B run?

solution

B1+A2

A: L(1) A1 U(1) L

B: L(1) B1 L(2) B2 U(2) U(1)

A: L(2) A2 U(2)

NOT A1+B2

would need to run B1 before A1 before A2 before B2

not possible because Lock1 held for entire B1+B2 operation

so cannot fit A1+A2 in between B1 and B2

exercise (alternate 1)

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
string one = "init one", two = "init two";
void ThreadA() {
    pthread_mutex_lock(&lock2);
    two = "two in ThreadA"; // (A2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_lock(&lock1);
    one = "one in ThreadA"; // (A1)
    pthread_mutex_unlock(&lock1);
}
void ThreadB() {
    pthread_mutex_lock(&lock1);
    one = "one in ThreadB"; // (B1)
    pthread_mutex_lock(&lock2);
    two = "two in ThreadB"; // (B2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}
```

possible values of one/two after A+B run?

exercise (alternate 2)

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
string one = "init one", two = "init two";
void ThreadA() {
    pthread_mutex_lock(&lock2);
    two = "two in ThreadA"; // (A2)
    pthread_mutex_unlock(&lock2);
    pthread_mutex_lock(&lock1);
    one = "one in ThreadA"; // (A1)
    pthread_mutex_unlock(&lock1);
}
void ThreadB() {
    pthread_mutex_lock(&lock1);
    one = "one in ThreadB"; // (B1)
    pthread_mutex_unlock(&lock1);
    pthread_mutex_lock(&lock2);
    two = "two in ThreadB"; // (B2)
    pthread_mutex_unlock(&lock2);
}
```

possible values of one/two after A+B run?

POSIX mutex restrictions

pthread_mutex rule: *unlock from same thread you lock in*

does this actually matter?

depends on how pthread_mutex is implemented

preview: general sync

lots of coordinating threads beyond locks

will talk about two general tools later:

- monitors/condition variables

- semaphores [if time]

big added feature: wait for arbitrary thing to happen

also some less general tools: barriers

a bad idea

one *bad* idea to wait for an event:

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER; bool ready = false;
void WaitForReady() {
    pthread_mutex_lock(&lock);
    do {
        pthread_mutex_unlock(&lock);
        /* only time MarkReady() can run */
        pthread_mutex_lock(&lock);
    } while (!ready);
    pthread_mutex_unlock(&lock);
}
void MarkReady() {
    pthread_mutex_lock(&lock);
    ready = true;
    pthread_mutex_unlock(&lock);
}
```

wastes processor time; MarkReady can stall waiting for unlock window

beyond locks

in practice: want more than locks for synchronization

for waiting for arbitrary events (without CPU-hogging-loop):

- monitors

- semaphores

for common synchronization patterns:

- barriers

- reader-writer locks

higher-level interface:

- transactions

barriers

compute minimum of 100M element array with 2 processors

algorithm:

compute minimum of 50M of the elements on each CPU

one thread for each CPU

wait for all computations to finish

take minimum of all the minimums

barriers

compute minimum of 100M element array with 2 processors

algorithm:

compute minimum of 50M of the elements on each CPU

one thread for each CPU

wait for all computations to finish

take minimum of all the minimums

barriers API

`barrier.Initialize(NumberOfThreads)`

`barrier.Wait()` — return after all threads have waited

idea: multiple threads perform computations in parallel

threads wait for *all other threads* to call `Wait()`

barrier: waiting for finish

```
barrier.Initialize(2);
```

Thread 0:

```
partial_mins[0] =  
    /* min of first  
       50M elems */;
```

```
barrier.Wait();
```

```
total_min = min(  
    partial_mins[0],  
    partial_mins[1]  
);
```

Thread 1:

```
partial_mins[1] =  
    /* min of last  
       50M elems */  
barrier.Wait();
```


barriers: reuse

```
results[0][0] = getInitial(0);  
barrier.Wait();
```

```
results[1][0] =  
    computeFrom(0,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][0] =  
    computeFrom(0,  
        results[1][0],  
        results[1][1]  
    );
```

```
results[0][1] = getInitial(1);  
barrier.Wait();
```

```
results[1][1] =  
    computeFrom(1,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][1] =  
    computeFrom(1,  
        results[1][0],  
        results[1][1]  
    );
```

barriers: reuse

```
results[0][0] = getInitial(0);  
barrier.Wait();
```

```
results[1][0] =  
    computeFrom(0,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][0] =  
    computeFrom(0,  
        results[1][0],  
        results[1][1]  
    );
```

```
results[0][1] = getInitial(1);  
barrier.Wait();
```

```
results[1][1] =  
    computeFrom(1,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][1] =  
    computeFrom(1,  
        results[1][0],  
        results[1][1]  
    );
```

barriers: reuse

```
results[0][0] = getInitial(0);  
barrier.Wait();
```

```
results[1][0] =  
    computeFrom(0,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][0] =  
    computeFrom(0,  
        results[1][0],  
        results[1][1]  
    );
```

```
results[0][1] = getInitial(1);  
barrier.Wait();
```

```
results[1][1] =  
    computeFrom(1,  
        results[0][0],  
        results[0][1]  
    );  
barrier.Wait();
```

```
results[2][1] =  
    computeFrom(1,  
        results[1][0],  
        results[1][1]  
    );
```

pthread barriers

```
pthread_barrier_t barrier;  
pthread_barrier_init(  
    &barrier,  
    NULL /* attributes */,  
    numberOfThreads  
);  
...  
...  
pthread_barrier_wait(&barrier);
```

exercise

```
pthread_barrier_t barrier; int x = 0, y = 0;
void thread_one() {
    y = 10;
    pthread_barrier_wait(&barrier);
    y = x + y;
    pthread_barrier_wait(&barrier);
    pthread_barrier_wait(&barrier);
    printf("%d %d\n", x, y);
}
void thread_two() {
    x = 20;
    pthread_barrier_wait(&barrier);
    pthread_barrier_wait(&barrier);
    x = x + y;
    pthread_barrier_wait(&barrier);
}
```

output? (if both run at once, barrier set for 2 threads)

life homework (pseudocode)

```
for (int time = 0; time < MAX_ITERATIONS; ++time) {  
    for (int y = 0; y < size; ++y) {  
        for (int x = 0; x < size; ++x) {  
            to_grid(x, y) = computeValue(from_grid, x, y);  
        }  
    }  
    swap(from_grid, to_grid);  
}
```

life homework

compute grid of values for time t from grid for time $t - 1$ parallel version: produce parts of grid in different threads use barriers to finish time t before going to time $t + 1$

modifying cache blocks in parallel

typical memory access — less than cache block

e.g. one 4-byte array element in 64-byte cache block

what if two processors modify different parts same cache block?

4-byte writes to 64-byte cache block

typically how caches work — write instructions happen one at a time:

processor 'locks' 64-byte cache block, fetching latest version

processor updates 4 bytes of 64-byte cache block

later, processor might give up cache block

modifying things in parallel (code)

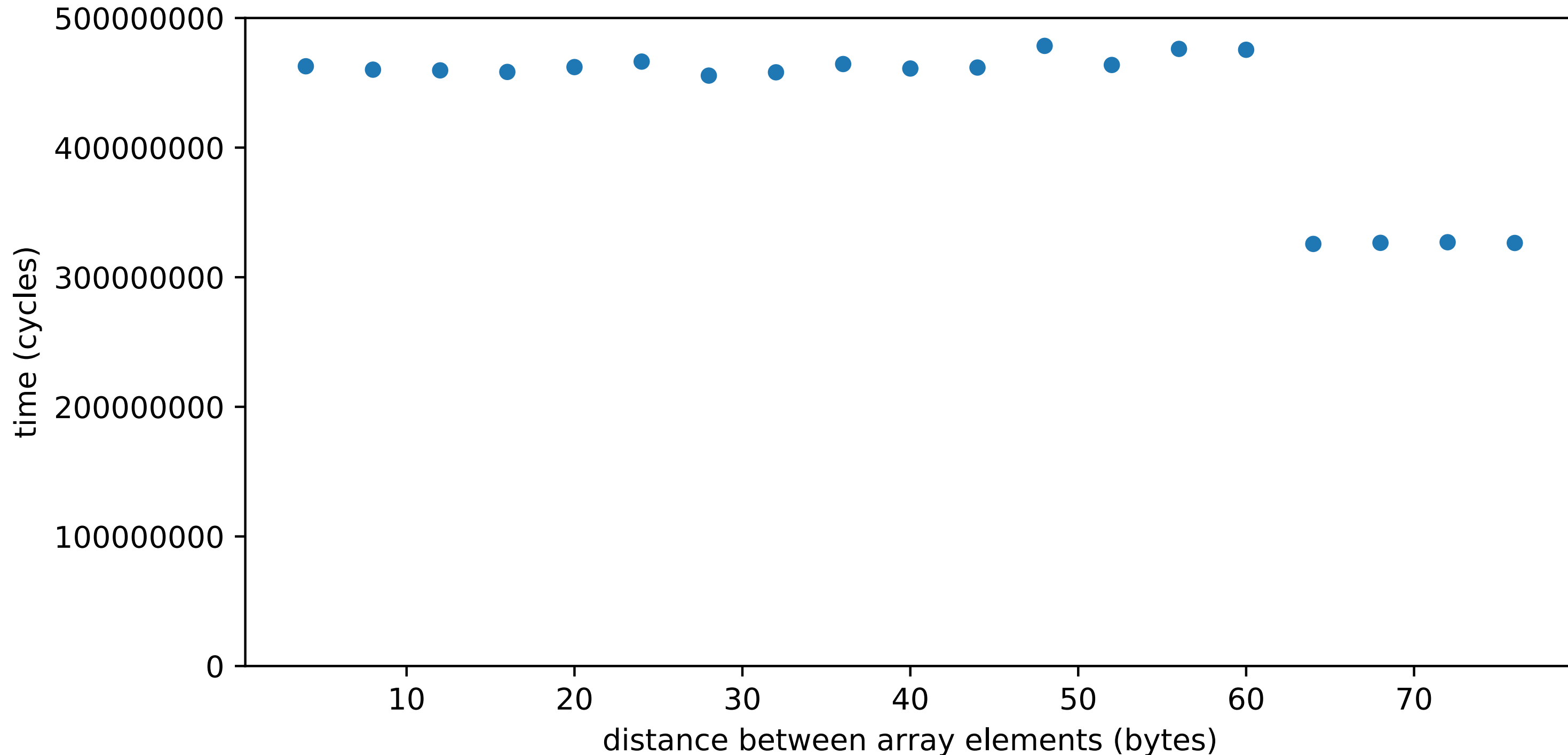
```
void *sum_up(void *raw_dest) {
    int *dest = (int *) raw_dest;
    for (int i = 0; i < 64 * 1024 * 1024; ++i) {
        *dest += data[i];
    }
}

__attribute__((aligned(4096)))
int dests[1024]; /* aligned = address is mult. of 4096 */

void sum_twice(int distance) {
    pthread_t threads[2];
    pthread_create(&threads[0], NULL, sum_up, &dests[0]);
    pthread_create(&threads[1], NULL, sum_up, &dests[distance]);
    pthread_join(threads[0], NULL);
    pthread_join(threads[1], NULL);
}
```

performance v. array element gap

(assuming `sum_up` compiled to not omit memory accesses)



false sharing

synchronizing to access two independent things

two parts of same cache block

solution: separate them

exercise (1)

```
int values[1024]; int results[2];
void *sum_front(void *ignored_argument) {
    results[0] = 0;
    for (int i = 0; i < 512; ++i)
        results[0] += values[i];
    return NULL;
}
void *sum_back(void *ignored_argument) {
    results[1] = 0;
    for (int i = 512; i < 1024; ++i)
        results[1] += values[i];
    return NULL;
}
int sum_all() {
    pthread_t sum_front_thread, sum_back_thread;
    pthread_create(&sum_front_thread, NULL, sum_front, NULL);
    pthread_create(&sum_back_thread, NULL, sum_back, NULL);
    pthread_join(sum_front_thread, NULL);
    pthread_join(sum_back_thread, NULL);
    return results[0] + results[1];
}
```

Where is false sharing likely to occur? How to fix?

exercise (2)

```
struct ThreadInfo { int *values; int start; int end; int result };
void *sum_thread(void *argument) {
    ThreadInfo *my_info = (ThreadInfo *) argument;
    int sum = 0;
    for (int i = my_info->start; i < my_info->end; ++i) {
        my_info->result += my_info->values[i];
    }
    return NULL;
}

int sum_all(int *values) {
    ThreadInfo info[2]; pthread_t thread[2];
    for (int i = 0; i < 2; ++i) {
        info[i].values = values; info[i].start = i*512; info[i].end = (i+1)*512;
        pthread_create(&threads[i], NULL, sum_thread, (void *) &info[i]);
    }
    for (int i = 0; i < 2; ++i)
        pthread_join(threads[i], NULL);
    return info[0].result + info[1].result;
}
```

Where is false sharing likely to occur now?

monitors/condition variables

locks for mutual exclusion

condition variables for waiting for event

represents *list of waiting threads*

operations: wait (for event); signal/broadcast (that event happened)

related data structures

monitor = lock + 0 or more condition variables + shared data

Java: every object is a monitor (has instance variables, built-in lock, cond. var)

pthread: build your own: provides you locks + condition variables

monitor idea

a monitor

lock
shared data
condvar 1
condvar 2
...
operation1(...)
operation2(...)

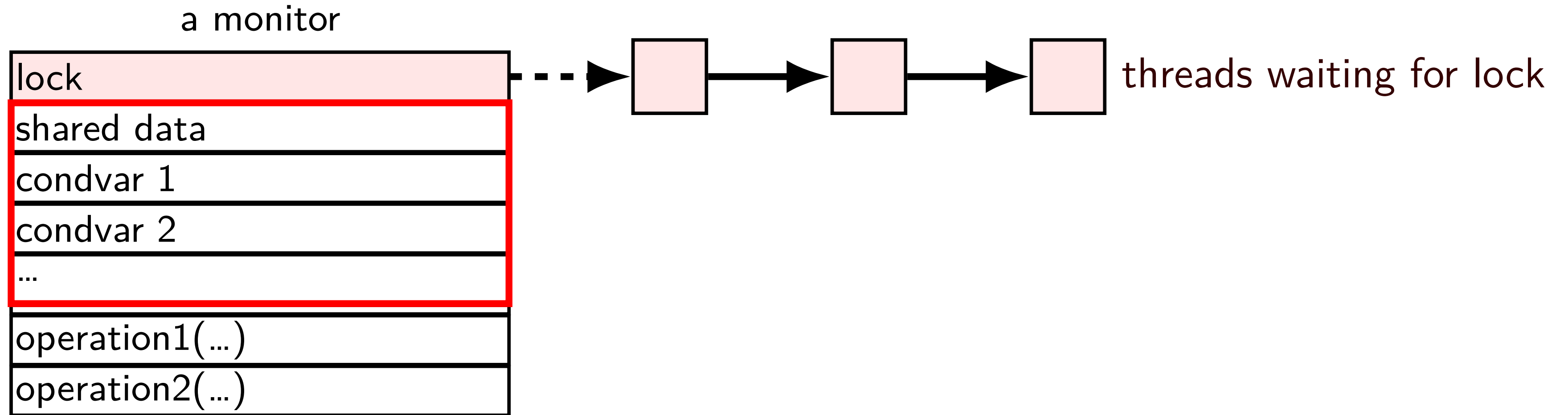
monitor idea

a monitor

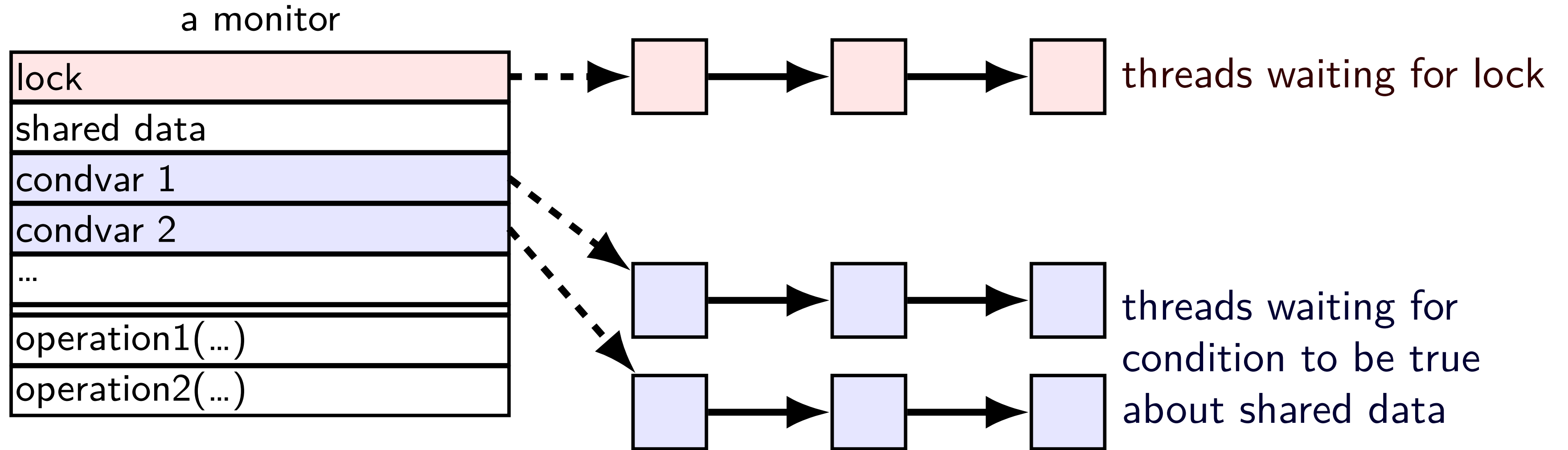
lock
shared data
condvar 1
condvar 2
...
operation1(...)
operation2(...)

lock must be acquired
before accessing
any part of monitor's stuff

monitor idea



monitor idea



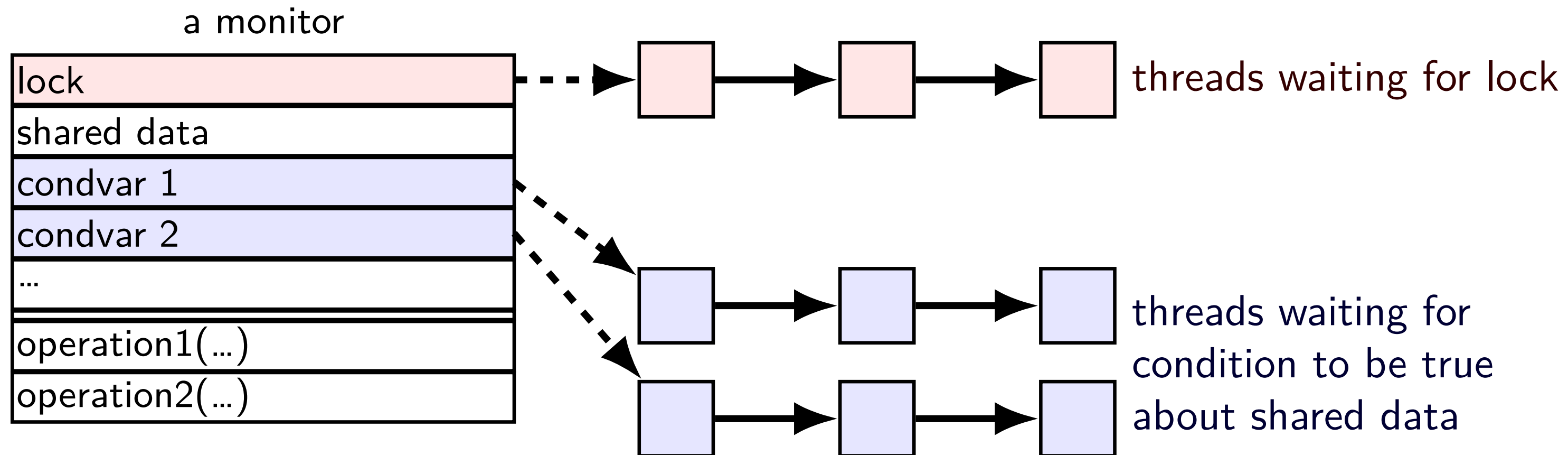
condvar operations

condvar operations:

Wait(cv, lock) — unlock lock, add current thread to cv queue
... and reacquire lock before returning

Broadcast(cv) — remove all from condvar queue

Signal(cv) — remove one from condvar queue



condvar operations

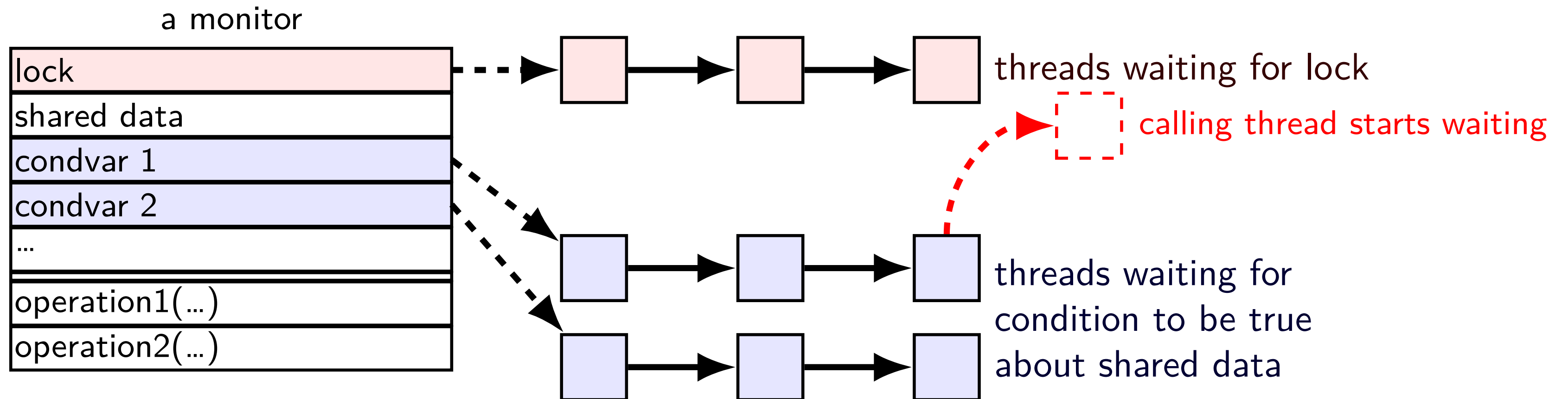
condvar operations:

Wait(cv, lock) — unlock lock, *add current thread* to cv queue

... and reacquire lock before returning

Broadcast(cv) — remove all from condvar queue

Signal(cv) — remove one from condvar queue



condvar operations

condvar operations:

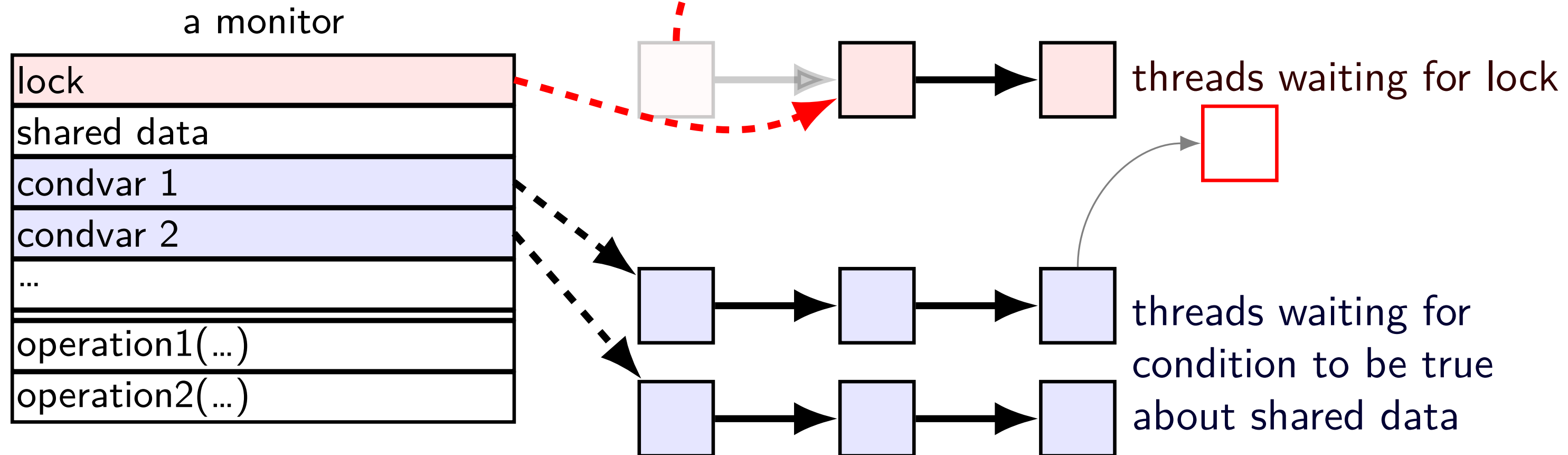
Wait(cv, lock) — *unlock* lock, add current thread to cv queue

... and *reacquire* lock before returning

Broadcast(cv) — remove all from condvar queue

Signal(cv) — remove one from condvar queue

unlock lock — allow thread from queue to go



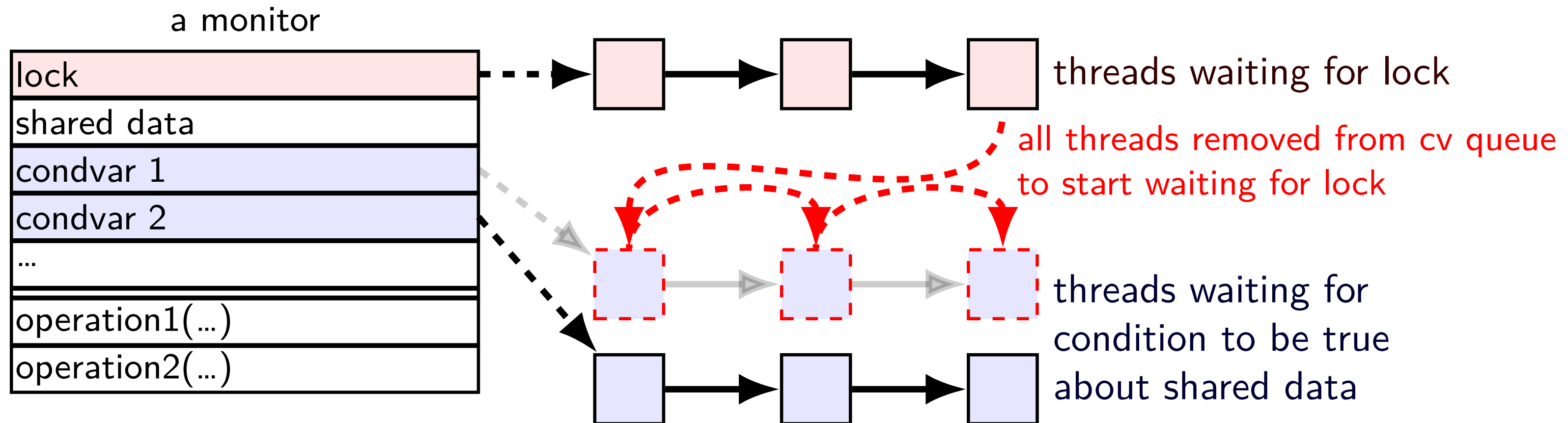
condvar operations

condvar operations:

Wait(cv, lock) — unlock lock, add current thread to cv queue
... and reacquire lock before returning

Broadcast(cv) — remove all from condvar queue

Signal(cv) — remove one from condvar queue



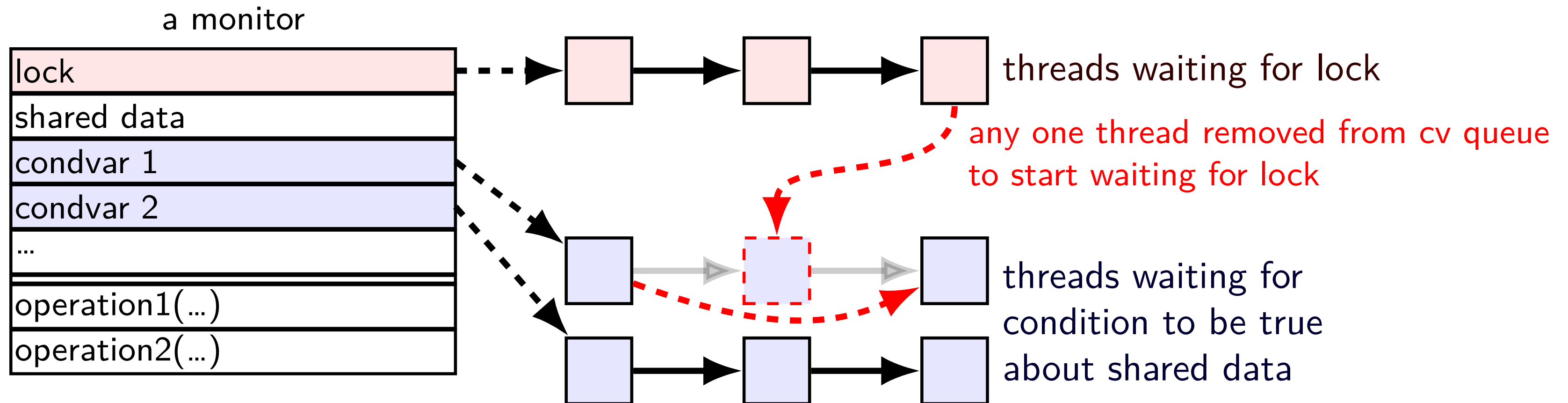
condvar operations

condvar operations:

Wait(cv, lock) — unlock lock, add current thread to cv queue
... and reacquire lock before returning

Broadcast(cv) — remove all from condvar queue

Signal(cv) — remove one from condvar queue



pthread cv usage

```
1 // MISSING: init calls, etc.
2 pthread_mutex_t lock;
3 bool finished; // data, only accessed with after acquiring lock
4 pthread_cond_t finished_cv; // to wait for 'finished' to be true
5
6 void WaitForFinished() {
7     pthread_mutex_lock(&lock);
8     while (!finished) {
9         pthread_cond_wait(&finished_cv, &lock);
10    }
11    pthread_mutex_unlock(&lock);
12 }
13
14 void Finish() {
15     pthread_mutex_lock(&lock);
16     finished = true;
17     pthread_cond_broadcast(&finished_cv);
18     pthread_mutex_unlock(&lock);
19 }
20
```


pthread cv usage — lock first

```
1 // MISSING: init calls, etc.
2 pthread_mutex_t lock;
3 bool finished; // data, only accessed with after acquiring lock
4 pthread_cond_t finished_cv; // to wait for 'finished' to be true
5
6 void WaitForFinished() {
7     pthread_mutex_lock(&lock);
8     while (!finished) {
9         pthread_cond_wait(&finished_cv, &lock);
10    }
11    pthread_mutex_unlock(&lock);
12 }
13
14 void Finish() {
15     pthread_mutex_lock(&lock);
16     finished = true;
17     pthread_cond_broadcast(&finished_cv);
18     pthread_mutex_unlock(&lock);
19 }
20
```

acquire lock before reading or write finished

pthread cv usage — check waiting?

```
1 // MISSING: init calls, etc.
2 pthread_mutex_t lock;
3 bool finished; // data, only accessed with after acquiring lock
4 pthread_cond_t finished_cv; // to wait for 'finished' to be true
5
6 void WaitForFinished() {
7     pthread_mutex_lock(&lock);
8     while (!finished) {
9         pthread_cond_wait(&finished_cv, &lock);
10    }
11    pthread_mutex_unlock(&lock);
12 }
13
14 void Finish() {
15     pthread_mutex_lock(&lock);
16     finished = true;
17     pthread_cond_broadcast(&finished_cv);
18     pthread_mutex_unlock(&lock);
19 }
20
```

check whether we need to wait at all
yes, must be a loop — we'll explain later

pthread cv usage — do waiting

```
1 // MISSING: init calls, etc.
2 pthread_mutex_t lock;
3 bool finished; // data, only accessed with after acquiring lock
4 pthread_cond_t finished_cv; // to wait for 'finished' to be true
5
6 void WaitForFinished() {
7     pthread_mutex_lock(&lock);
8     while (!finished) {
9         pthread_cond_wait(&finished_cv, &lock);
10    }
11    pthread
12 }
13
14 void Fini
15 pthread
16 finishe
17 pthread
18 pthread
19 }
20
```

know we need to wait

(finished cannot have changed since we checked because of lock)

so wait releasing lock

important that we release lock, so finished can change

pthread cv usage — broadcast

```
1 // MISSING: init calls, etc.
2 pthread_mutex_t lock;
3 bool finished; // data, only accessed with after acquiring lock
4 pthread_cond_t finished_cv; // to wait for 'finished' to be true
5
6 void WaitForFinished() {
7     pthread_mutex_lock(&lock);
8     while (!finished) {
9         pthread_cond_wait(&finished_cv, &lock);
10    }
11    pthread_mutex_unlock(&lock);
12 }
13
14 void Finish() {
15     pthread_mutex_lock(&lock);
16     finished = true;
17     pthread_cond_broadcast(&finished_cv);
18     pthread_mutex_unlock(&lock);
19 }
20
```

all waiters to proceed once we release lock

WaitForFinish timeline 1

WaitForFinish thread

`mutex_lock(&lock)`
(thread has lock)

`while (!finished) ...`
`cond_wait(&finished_cv, &lock);`
(start waiting for cv)

(done waiting for cv)
(start waiting for lock)

(done waiting for lock)
`while (!finished) ...`
(finished now true, so return)
`mutex_unlock(&lock)`

Finish thread

`mutex_lock(&lock)`
(start waiting for lock)

(done waiting for lock)
`finished = true`
`cond_broadcast(&finished_cv)`

`mutex_unlock(&lock)`

WaitForFinish timeline 2

WaitForFinish thread

```
mutex_lock(&lock)
while (!finished) ...
(finished now true, so return)
mutex_unlock(&lock)
```

Finish thread

```
mutex_lock(&lock)
finished = true
cond_broadcast(&finished_cv)
mutex_unlock(&lock)
```

why the loop

```
while (!finished) {  
    pthread_cond_wait(&finished_cv, &lock);  
}
```

we only broadcast if finished is true

so why check finished afterwards?

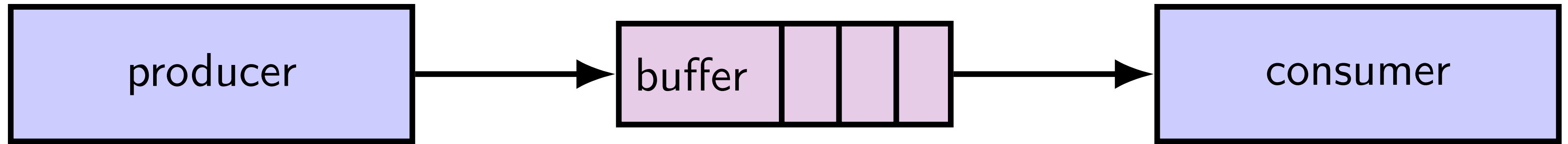
pthread_cond_wait manual page:

“*Spurious wakeups* ... may occur.”

spurious wakeup = wait returns even though nothing happened

producer/consumer idea

example: producer/consumer



shared buffer (queue) of fixed size

- one or more producers inserts into queue

- one or more consumers removes from queue

producer(s) and consumer(s) don't work in lockstep
(might need to wait for each other to catch up)

example: C compiler

- preprocessor → compiler → assembler → linker

unbounded buffer producer/consumer

```
1  pthread_mutex_t lock;
2  pthread_cond_t data_ready;
3  Queue buffer;
4  Produce(item) {
5      pthread_mutex_lock(&lock);
6      buffer.enqueue(item);
7      pthread_cond_signal(&data_ready);
8      pthread_mutex_unlock(&lock);
9  }
10 Consume() {
11     pthread_mutex_lock(&lock);
12     while (buffer.empty()) {
13         pthread_cond_wait(&data_ready, &lock);
14     }
15     item = buffer.dequeue();
16     pthread_mutex_unlock(&lock);
17     return item;
18 }
19
```

unbounded p/c — lock

```
1  pthread_mutex_t lock;
2  pthread_cond_t data_ready;
3  Queue buffer;
4  Produce(item) {
5      pthread_mutex_lock(&lock);
6      buffer.enqueue(item);
7      pthread_cond_signal(&data_ready);
8      pthread_mutex_unlock(&lock);
9  }
10 Consume() {
11     pthread_mutex_lock(&lock);
12     while (buffer.empty()) {
13         pthread_cond_wait(&data_ready, &lock);
14     }
15     item = buffer.dequeue();
16     pthread_mutex_unlock(&lock);
17     return item;
18 }
19
```

rule: never touch buffer without acquiring lock

otherwise: what if two threads simulatenously en/dequeue?

unbounded p/c — dequeue if empty

```
1  pthread_mutex_t lock;
2  pthread_cond_t data_ready;
3  Queue buffer;
4  Produce(item) {
5      pthread_mutex_lock(&lock);
6      buffer.enqueue(item);
7      pthread_cond_signal(&data_ready);
8      pthread_mutex_unlock(&lock);
9  }
10 Consume() {
11     pthread_mutex_lock(&lock);
12     while (buffer.empty()) {
13         pthread_cond_wait(&data_ready, &lock);
14     }
15     item = buffer.dequeue();
16     pthread_mutex_unlock(&lock);
17     return item;
18 }
19
```

check if empty, then dequeue

(note: have lock — don't need to worry about someone else dequeuing)

unbounded p/c — wake

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 Queue buffer;
4 Produce(item) {
5     pthread_mutex_lock(&lock);
6     buffer.enqueue(item);
7     pthread_cond_signal(&data_ready);
8     pthread_mutex_unlock(&lock);
9 }
10 Consume() {
11     pthread_mutex_lock(&lock);
12     while (buffer.empty()) {
13         pthread_cond_wait(&data_ready, &lock);
14     }
15     item = buffer.dequeue();
16     pthread_mutex_unlock(&lock);
17     return item;
18 }
19
```

check if empty (waiting if needed), then dequeue
(note: have lock — don't need to worry about someone else dequeuing)

unbounded p/c — wake

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 Queue buffer;
4 Produce(item) {
5     pthread_mutex_lock(&lock);
6     buffer.enqueue(item);
7     pthread_cond_signal(&data_ready);
8     pthread_mutex_unlock(&lock);
9 }
10 Consume() {
11     pthread_mutex_lock(&lock);
12     while (buffer.empty()) {
13         pthread_cond_wait(&data_ready, &lock);
14     }
15     item = buffer.dequeue();
16     pthread_mutex_unlock(&lock);
17     return item;
18 }
19
```

after enqueueing
wake one Consume thread (if any are waiting)

unbounded p/c — waiting

```
1  pthread_mutex_t lock;
2  pthread_cond_t data_ready;
3  Queue buffer;
4  Produce(item) {
5      pthread_mutex_lock(&lock);
6      buffer.enqueue(item);
7      pthread_cond_signal(&data_ready);
8      pthread_mutex_unlock(&lock);
9  }
10 Consume() {
11     pthread_mutex_lock(&lock);
12     while (buffer.empty()) {
13         pthread_cond_wait(&data_ready, &lock);
14     }
15     item = buffer.dequeue();
16     pthread_mutex_unlock(&lock);
17     return item;
18 }
19
```

unbounded p/c — waiting

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 Queue buffer;
4 Produce(item) {
5     pthread_mutex_lock(&lock);
6     buffer.enqueue(item);
7     pthread_cond_signal(&data_ready);
8     pthread_mutex_unlock(&lock);
9 }
10 Consume() {
11     pthread_mutex_lock(&lock);
12     while (buffer.empty()) {
13         pthread_cond_wait(&data_ready, &lock);
14     }
15     item = buffer.dequeue();
16     pthread_mutex_unlock(&lock);
17     return item;
18 }
19
```

while loop could run 0 times

Thread 1

Thread 2

Produce()
... lock
... enqueue
... signal
... unlock

Consume?
... lock
... empty? no
... dequeue
... unlock
return

unbounded p/c — waiting

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 Queue buffer;
4 Produce(item) {
5     pthread_mutex_lock(&lock);
6     buffer.enqueue(item);
7     pthread_cond_signal(&data_ready);
8     pthread_mutex_unlock(&lock);
9 }
10 Consume() {
11     pthread_mutex_lock(&lock);
12     while (buffer.empty()) {
13         pthread_cond_wait(&data_ready, &lock);
14     }
15     item = buffer.dequeue();
16     pthread_mutex_unlock(&lock);
17     return item;
18 }
19
```

while loop could run 1 times

Thread 1	Thread 2
Produce() ... lock ... enqueue ... signal ... unlock	Consume? ... lock ... empty? yes ... unlock/start wait stop wait lock ... empty? no ... dequeue ... unlock return

unbounded p/c — waiting

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 Queue buffer;
4 Produce(item) {
5     pthread_mutex_lock(&lock);
6     buffer.enqueue(item);
7     pthread_cond_signal(&data_ready);
8     pthread_mutex_unlock(&lock);
9 }
10 Consume() {
11     pthread_mutex_lock(&lock);
12     while (buffer.empty()) {
13         pthread_cond_wait(&data_ready, &lock);
14     }
15     item = buffer.dequeue();
16     pthread_mutex_unlock(&lock);
17     return item;
18 }
19
```

while loop could run 2+ times

Thread 1	Thread 2	Thread 3
	Consume? ... lock ... empty? yes ... unlock/start wait	
Produce() ... lock ... enqueue ... signal ... unlock	stop wait lock	
		Consume? ... lock ... empty? no ... dequeue ... unlock return
	... empty? no ... dequeue ... unlock return	

Hoare versus Mesa monitors

Hoare-style monitors

- signal 'hands off' lock to awoken thread

Mesa-style monitors

- any eligible thread gets lock next

- (maybe some other idea of priority?)

every current threading library I know of does Mesa-style

bounded buffer producer/consumer — full code

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 pthread_cond_t space_ready;
4 BoundedQueue buffer;
5 Produce(item) {
6     pthread_mutex_lock(&lock);
7     while (buffer.full()) {
8         pthread_cond_wait(&space_ready, &lock);
9     }
10    buffer.enqueue(item);
11    pthread_cond_signal(&data_ready);
12    pthread_mutex_unlock(&lock);
13 }
14 Consume() {
15     pthread_mutex_lock(&lock);
16     while (buffer.empty()) {
17         pthread_cond_wait(&data_ready, &lock);
18     }
19     item = buffer.dequeue();
20     pthread_cond_signal(&space_ready);
21     pthread_mutex_unlock(&lock);
22     return item;
23 }
24
```

bounded buffer p/c— added

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 pthread_cond_t space_ready;
4 BoundedQueue buffer;
5 Produce(item) {
6     pthread_mutex_lock(&lock);
7     while (buffer.full()) {
8         pthread_cond_wait(&space_ready, &lock);
9     }
10    buffer.enqueue(item);
11    pthread_cond_signal(&data_ready);
12    pthread_mutex_unlock(&lock);
13 }
14 Consume() {
15     pthread_mutex_lock(&lock);
16     while (buffer.empty()) {
17         pthread_cond_wait(&data_ready, &lock);
18     }
19     item = buffer.dequeue();
20     pthread_cond_signal(&space_ready);
21     pthread_mutex_unlock(&lock);
22     return item;
23 }
24
```


bounded buffer p/c— added

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 pthread_cond_t space_ready;
4 BoundedQueue buffer;
5 Produce(item) {
6     pthread_mutex_lock(&lock);
7     while (buffer.full()) {
8         pthread_cond_wait(&space_ready, &lock);
9     }
10    buffer.enqueue(item);
11    pthread_cond_signal(&data_ready);
12    pthread_mutex_unlock(&lock);
13 }
14 Consume() {
15     pthread_mutex_lock(&lock);
16     while (buffer.empty()) {
17         pthread_cond_wait(&data_ready, &lock);
18     }
19    item = buffer.dequeue();
20    pthread_cond_signal(&space_ready);
21    pthread_mutex_unlock(&lock);
22    return item;
23 }
24
```

add new condition variable
for new reason to wait

wait on that condition
while reason to wait is true

signal that condition
when one thread can stop waiting

bounded buffer p/c — signal/broadcast

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 pthread_cond_t space_ready;
4 BoundedQueue buffer;
5 Produce(item) {
6     pthread_mutex_lock(&lock);
7     while (buffer.full()) {
8         pthread_cond_wait(&space_ready, &lock);
9     }
10    buffer.enqueue(item);
11    pthread_cond_signal(&data_ready);
12    pthread_mutex_unlock(&lock);
13 }
14 Consume() {
15     pthread_mutex_lock(&lock);
16     while (buffer.empty()) {
17         pthread_cond_wait(&data_ready, &lock);
18     }
19     item = buffer.dequeue();
20     pthread_cond_signal(&space_ready);
21     pthread_mutex_unlock(&lock);
22     return item;
23 }
24
```

bounded buffer p/c — signal/broadcast

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 pthread_cond_t space_ready;
4 BoundedQueue buffer;
5 Produce(item) {
6     pthread_mutex_lock(&lock);
7     while (buffer.isFull()) {
8         pthread_cond_wait(&space_ready, &lock);
9     }
10    buffer.enqueue(item);
11    pthread_cond_signal(&data_ready);
12    pthread_mutex_unlock(&lock);
13 }
14 Consume() {
15     pthread_mutex_lock(&lock);
16     while (buffer.isEmpty()) {
17         pthread_cond_wait(&data_ready, &lock);
18     }
19     item = buffer.dequeue();
20     pthread_cond_signal(&space_ready);
21     pthread_mutex_unlock(&lock);
22     return item;
23 }
24
```

signal better than broadcast in this case
only one waiting thread can do something useful
correct (but slow) to broadcast instead

bounded buffer p/c — signal/broadcast

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 pthread_cond_t space_ready;
4 BoundedQueue buffer;
5 Produce(item) {
6     pthread_mutex_lock(&lock);
7     while (buffer.full()) {
8         pthread_cond_wait(&space_ready, &lock);
9     }
10    buffer.enqueue(item);
11    pthread_cond_signal(&data_ready);
12    pthread_mutex_unlock(&lock);
13 }
14 Consume() {
15     pthread_mutex_lock(&lock);
16     while (buffer.empty()) {
17         pthread_cond_wait(&data_ready, &lock);
18     }
19     item = buffer.dequeue();
20     pthread_cond_signal(&space_ready);
21     pthread_mutex_unlock(&lock);
22     return item;
23 }
24
```

bounded buffer p/c — signal/broadcast

```
1 pthread_mutex_t lock;
2 pthread_cond_t data_ready;
3 pthread_cond_t space_ready;
4 BoundedQueue buffer;
5 Produce(item) {
6     pthread_mutex_lock(&lock);
7     while (buffer.full()) {
8         pthread_cond_wait(&space_ready, &lock);
9     }
10    buffer.enqueue(item);
11    pthread_cond_signal(&data_ready);
12    pthread_mutex_unlock(&lock);
13 }
14 Consume() {
15     pthread_mutex_lock(&lock);
16     while (buffer.empty()) {
17         pthread_cond_wait(&data_ready, &lock);
18     }
19    item = buffer.dequeue();
20    pthread_cond_signal(&space_ready);
21    pthread_mutex_unlock(&lock);
22    return item;
23 }
24
```

correct but slow to use just one
condition variable:

data_ready, space_ready $\Rightarrow$ ready

always use broadcast

let each thread figure out if it was supposed
to be woken up

monitor pattern

```
pthread_mutex_lock(&lock);
while (!condition A) {
    pthread_cond_wait(&condvar_for_A, &lock);
}
... /* manipulate shared data, changing other conditions */
if (set condition A) {
    pthread_cond_broadcast(&condvar_for_A);
    /* or signal, if only one thread cares */
}
if (set condition B) {
    pthread_cond_broadcast(&condvar_for_B);
    /* or signal, if only one thread cares */
}
...
pthread_mutex_unlock(&lock)
```

monitors rules of thumb

never touch shared data without holding the lock

keep lock held for *entire operation*:

- verifying condition (e.g. buffer not full) *up to and including*
- manipulating data (e.g. adding to buffer)

create condvar for every kind of scenario waited for

always write *loop* calling cond_wait to wait for condition X

broadcast/signal condition variable *every time you change X*

correct but slow to...

- broadcast when just signal would work

- broadcast or signal when nothing changed

- use one condvar for multiple conditions

mutex/cond var init/destroy

```
pthread_mutex_t mutex;  
pthread_cond_t cv;  
pthread_mutex_init(&mutex, NULL);  
pthread_cond_init(&cv, NULL);  
// --OR--  
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;  
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;  
  
// and when done:  
...  
pthread_cond_destroy(&cv);  
pthread_mutex_destroy(&mutex);
```


wait for both finished (0)

```
1 // MISSING: init calls, etc.
2 pthread_mutex_t lock;
3 bool finished[2];
4 pthread_cond_t both_finished_cv;
5
6 void WaitForBothFinished() {
7     pthread_mutex_lock(&lock);
8     while (_____) {
9         pthread_cond_wait(&both_finished_cv, &lock);
10    }
11    pthread_mutex_unlock(&lock);
12 }
13
14 void Finish(int index) {
15     pthread_mutex_lock(&lock);
16     finished[index] = true;
17     _____
18     pthread_mutex_unlock(&lock);
19 }
20
```

wait for both finished (1)

```
1 // MISSING: init calls, etc.
2 pthread_mutex_t lock;
3 bool finished[2];
4 pthread_cond_t both_finished_cv;
5
6 void WaitForBothFinished() {
7     pthread_mutex_lock(&lock);
8     while (_____) {
9         pthread_cond_wait(&both_finished_cv, &lock);
10    }
11    pthread_mutex_unlock(&lock);
12 }
13
14 void Finish(int index) {
15     pthread_mutex_lock(&lock);
16     finished[index] = true;
17     _____
18     pthread_mutex_unlock(&lock);
19 }
20
```

- A. `finished[0] && finished[1]`
- B. `finished[0] || finished[1]`
- C. `!finished[0] || !finished[1]`
- D. `finsihed[0] != finished[1]`
- E. something else

wait for both finished (?)

```
1 // MISSING: init calls,
2 pthread_mutex_t lock;
3 bool finished[2];
4 pthread_cond_t both_finished;
5
6 void WaitForBothFinished() {
7     pthread_mutex_lock(&lock);
8     while (true) {
9         pthread_cond_wait(&both_finished, &lock);
10    }
11    pthread_mutex_unlock(&lock);
12 }
13
14 void Finish(int index) {
15     pthread_mutex_lock(&lock);
16     finished[index] = true;
17     // MISSING: signal both_finished
18     pthread_mutex_unlock(&lock);
19 }
20
```

A. `pthread_cond_signal(&both_finished_cv)`

B. `pthread_cond_broadcast(&both_finished_cv)`

C. `if (finished[1-index])`

`pthread_cond_signal(&both_finished_cv);`

D. `if (finished[1-index])`

`pthread_cond_broadcast(&both_finished_cv);`

E. something else

monitor exercise: one-use barrier

suppose we want to implement a one-use barrier; fill in blanks:

```
struct BarrierInfo {
    pthread_mutex_t lock;
    int total_threads; // initially total # of threads
    int number_reached; // initially 0
    -----
};
void BarrierWait(BarrierInfo *b) {
    pthread_mutex_lock(&b->lock);
    ++b->number_reached;
    if (b->number_reached == b->total_threads) {
        -----
    } else {
        -----
        -----
    }
    pthread_mutex_unlock(&b->lock);
}
```

monitor exercise: one-use barrier

```
struct BarrierInfo {  
    pthread_mutex_t lock;  
    int total_threads; // initially total # of threads  
    int number_reached; // initially 0  
    pthread_cond_t cv;  
};  
  
void BarrierWait(BarrierInfo *b) {  
    pthread_mutex_lock(&b->lock);  
    ++b->number_reached;  
    if (b->number_reached == b->total_threads) {  
        pthread_cond_broadcast(&b->cv);  
    } else {  
        while (b->number_reached < b->total_threads)  
            pthread_cond_wait(&b->cv, &b->lock);  
    }  
    pthread_mutex_unlock(&b->lock);  
}
```

reader/writer problem

some shared data

only one thread modifying (read+write) at a time

read-only access *from multiple threads* is safe

could use lock — but doesn't allow multiple readers

reader/writer locks

abstraction: lock that distinguishes readers/writers

operations:

- read lock: wait until no writers

- read unlock: stop being registered as reader

- write lock: wait until no readers and no writers

- write unlock: stop being registered as writer

reader/writer locks

abstraction: lock that distinguishes readers/writers

operations:

- read lock: wait until no writers

- read unlock: stop being registered as reader

- write lock: wait until *no readers and no writers*

- write unlock: stop being registered as writer

pthread_rwlock_t

```
pthread_rwlock_t rwlock;
pthread_rwlock_init(&rwlock, NULL /* attributes */);
...
    pthread_rwlock_rdlock(&rwlock);
    ... /* read shared data */
    pthread_rwlock_unlock(&rwlock);

    pthread_rwlock_wrlock(&rwlock);
    ... /* read+write shared data */
    pthread_rwlock_unlock(&rwlock);

...
pthread_rwlock_destroy(&rwlock);
```

rwlock effects exercise

```
pthread_rwlock_t lock;
void ThreadA() {
    pthread_rwlock_rdlock(&lock);
    puts("a");
    ...
    puts("A");
    pthread_rwlock_unlock(&lock);
}
void ThreadB() {
    pthread_rwlock_rdlock(&lock);
    puts("b");
    ...
    puts("B");
    pthread_rwlock_unlock(&lock);
}
```

```
void ThreadC() {
    pthread_rwlock_wrlock(&lock);
    puts("c");
    ...
    puts("C");
    pthread_rwlock_unlock(&lock);
}
void ThreadD() {
    pthread_rwlock_wrlock(&lock);
    puts("d");
    ...
    puts("D");
    pthread_rwlock_unlock(&lock);
}
```

exercise: which of these outputs are possible?

1. aAbBcCdD 2. abABcdDC 3. cCabBAdD
4. cdCDaAbB 5. caACdDbB

generalizing locks: semaphores

semaphore has a non-negative integer *value* and two operations:

P() or *down* or *wait*:

wait for semaphore to become positive (> 0),
then decrement by 1

V() or *up* or *signal* or *post*:

increment semaphore by 1 (waking up thread if needed)

P, V from Dutch: *proberen* (test), *verhogen* (increment)

semaphores are kinda integers

semaphore like an integer, but...

cannot read/write directly

down/up operation only way to access (typically)

exception: initialization

never negative — wait instead

down operation wants to make negative? thread waits

reserving books

suppose tracking copies of library book...

```
Semaphore free_copies = Semaphore(3);  
void ReserveBook() {  
    // wait for copy to be free  
    free_copies.down();  
    ... // ... then take reserved copy  
}  
  
void ReturnBook() {  
    ... // return reserved copy  
    free_copies.up();  
    // ... then wakeup waiting thread  
}
```

counting resources: reserving books

suppose tracking copies of same library book

non-negative integer count = # how many books used?

up = give back book; down = take book

Copy 1
Copy 2
Copy 3

free copies 3

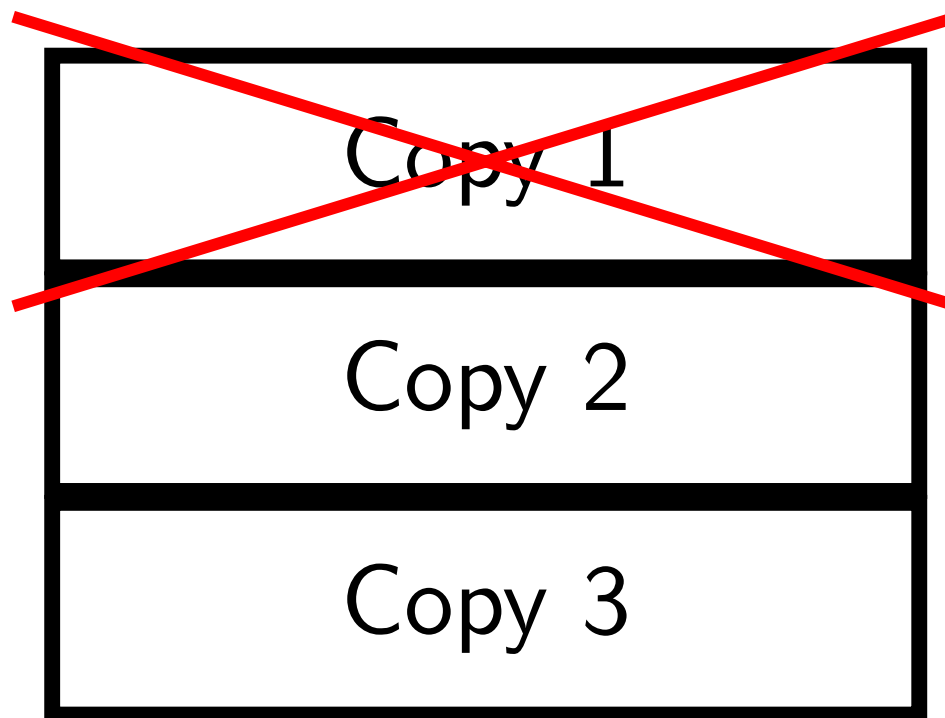
counting resources: reserving books

suppose tracking copies of same library book

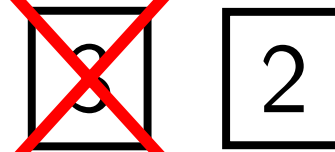
non-negative integer count = # how many books used?

up = give back book; down = take book

taken out



free copies



after calling down to reserve

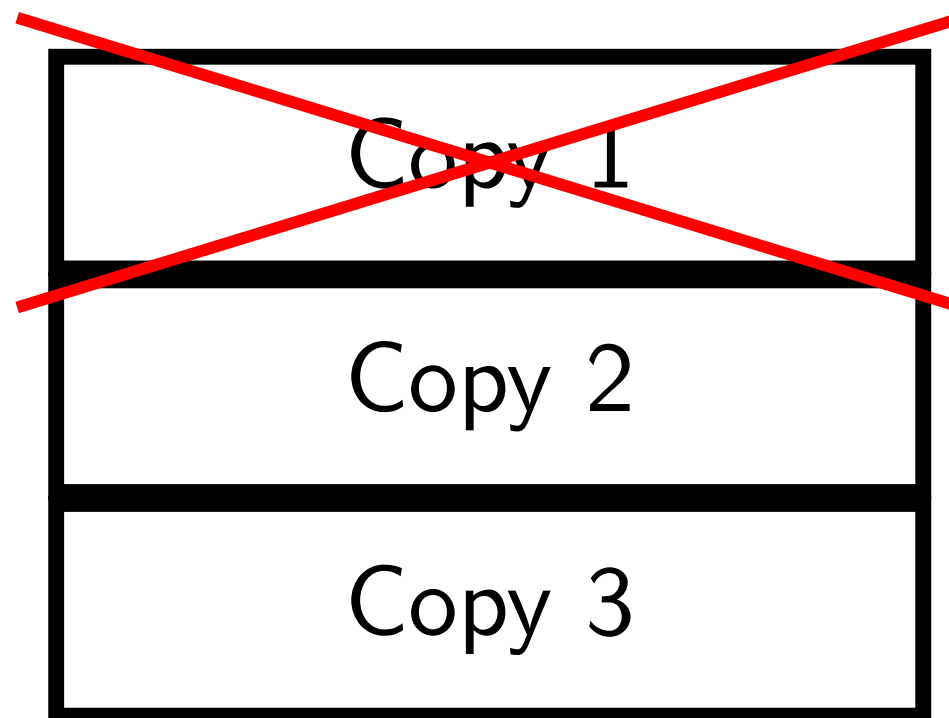
counting resources: reserving books

suppose tracking copies of same library book

non-negative integer count = # how many books used?

up = give back book; down = take book

taken out



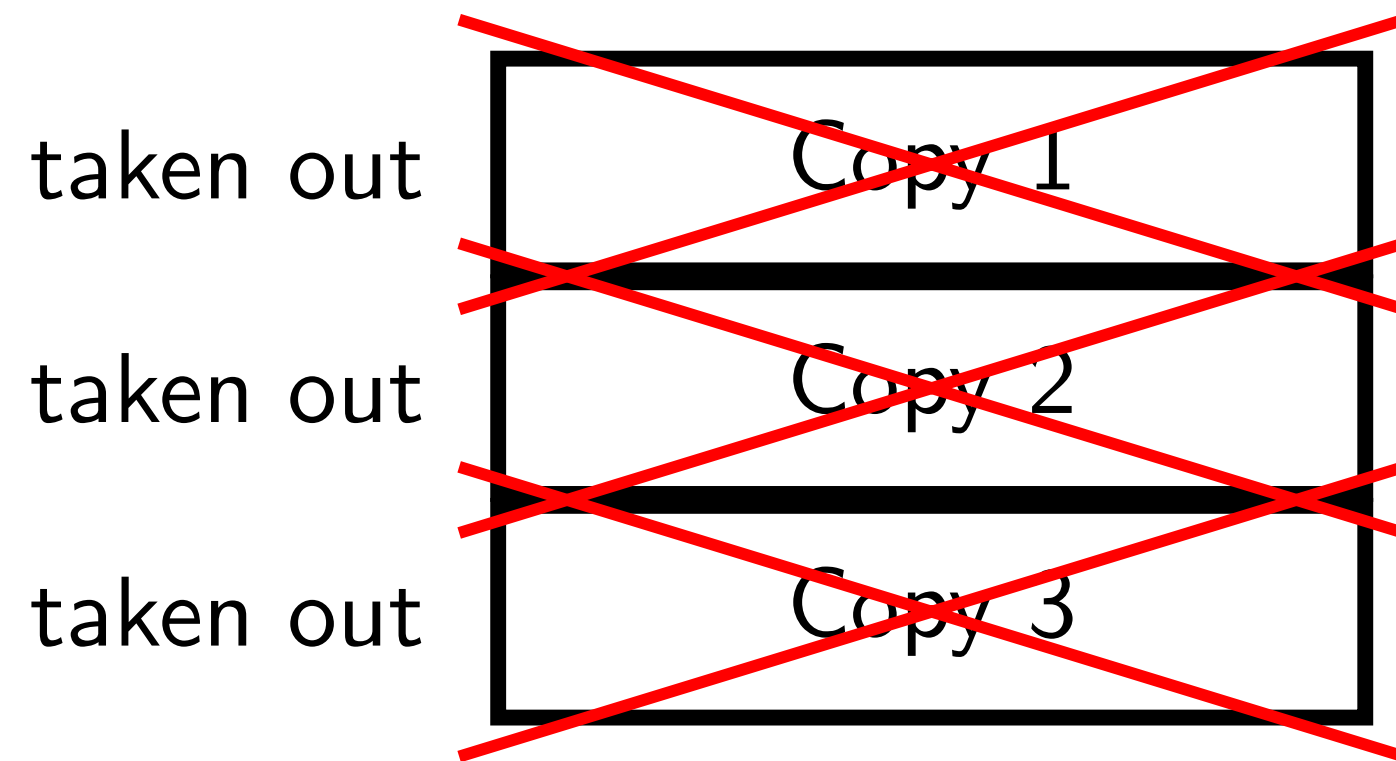
free copies 2
after calling down to reserve

counting resources: reserving books

suppose tracking copies of same library book

non-negative integer count = # how many books used?

up = give back book; down = take book



free copies 0

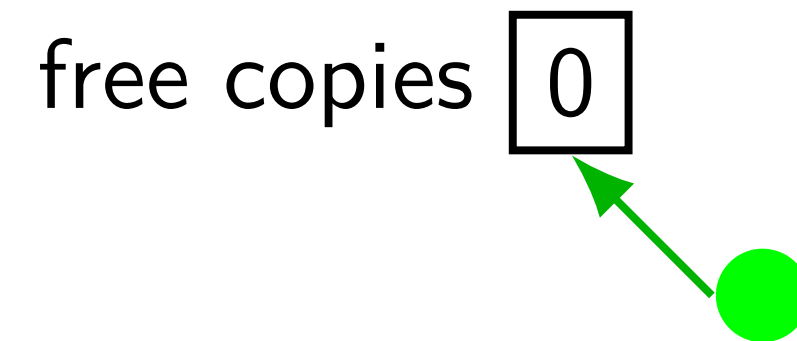
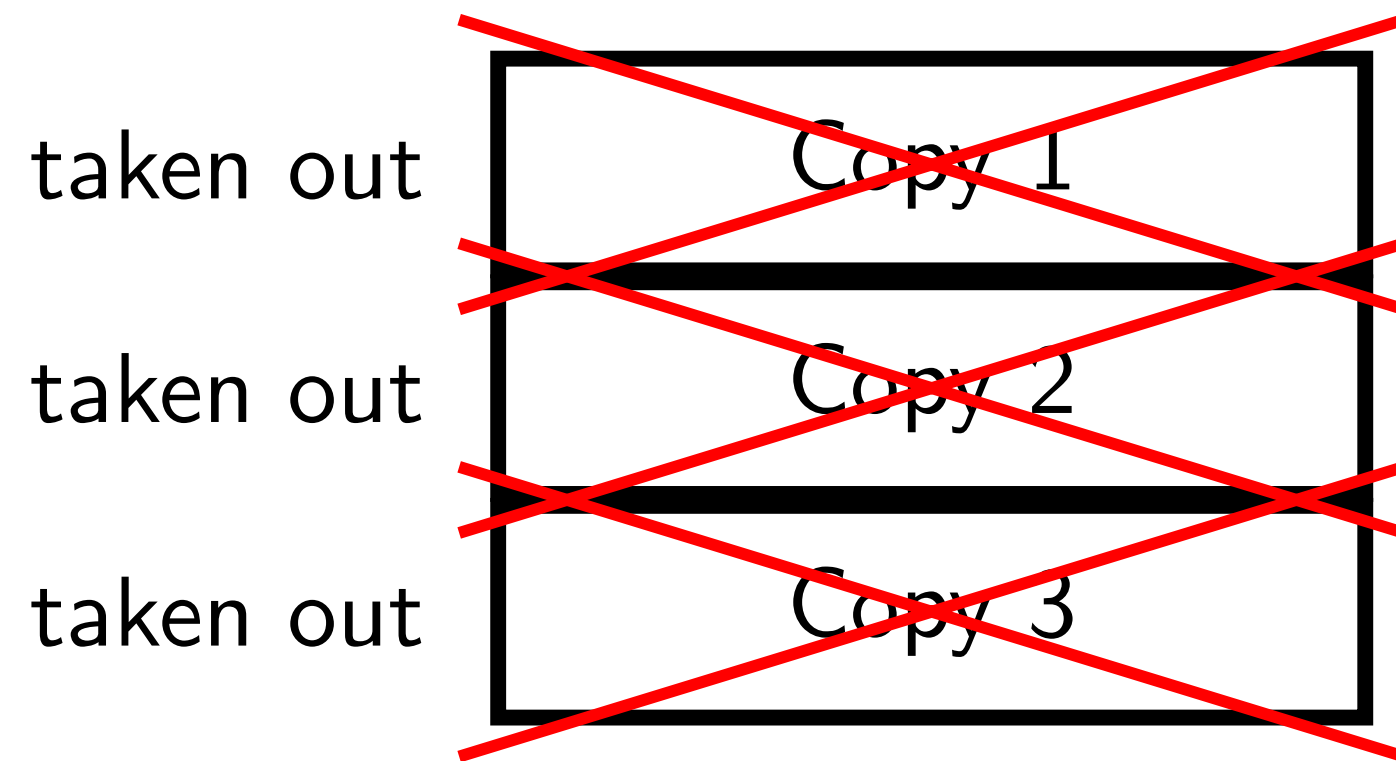
after calling **down** three times
to reserve all copies

counting resources: reserving books

suppose tracking copies of same library book

non-negative integer count = # how many books used?

up = give back book; **down** = take book



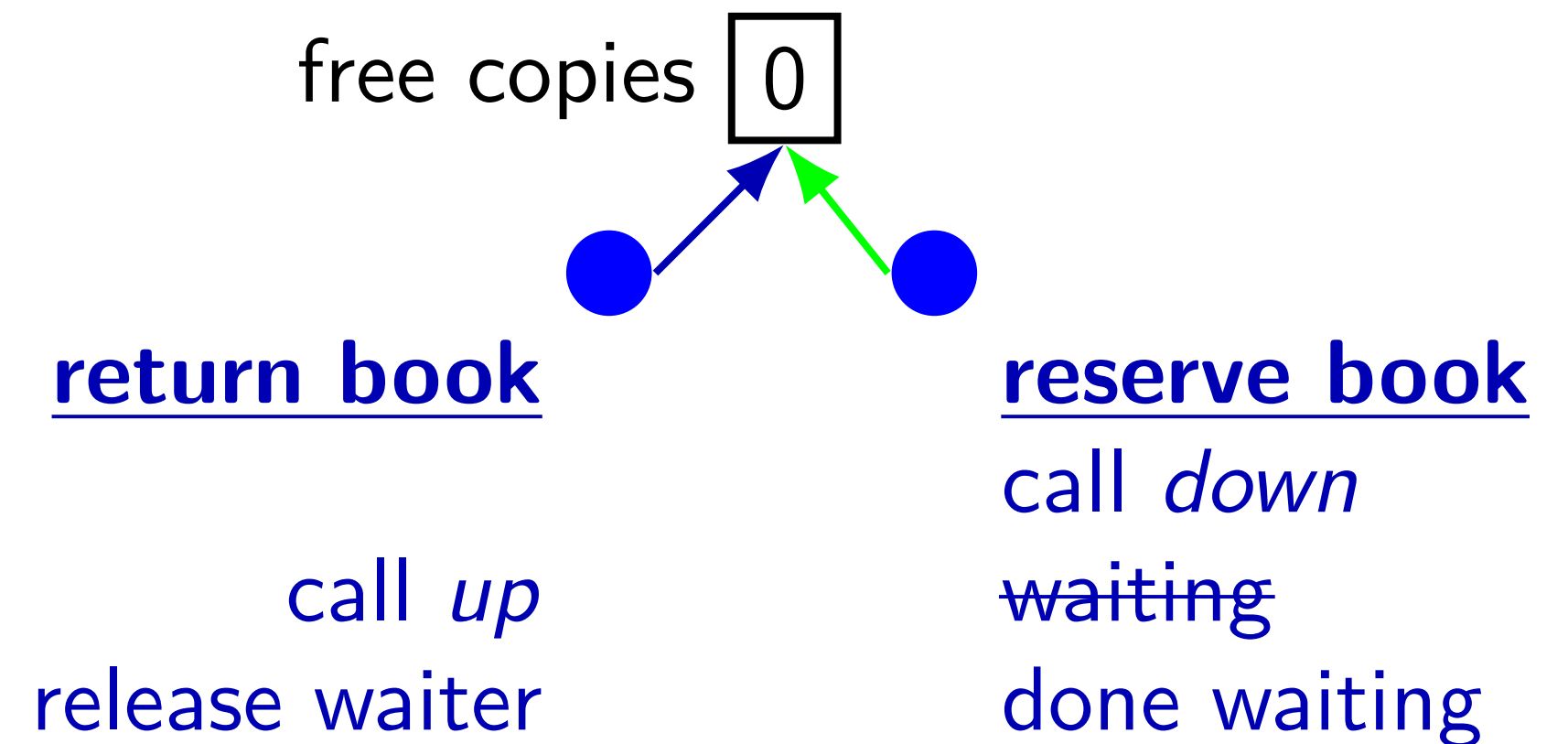
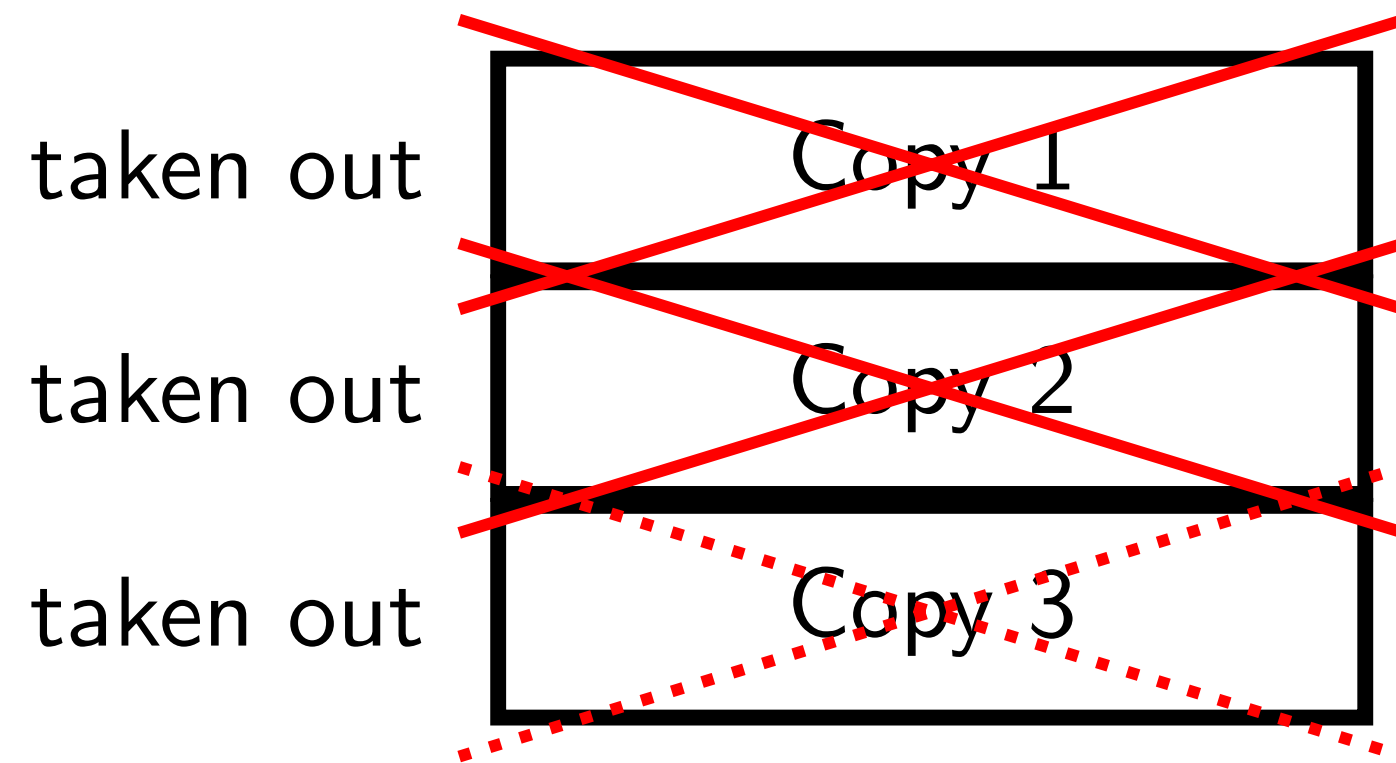
reserve book
call *down* again
start waiting...

counting resources: reserving books

suppose tracking copies of same library book

non-negative integer count = # how many books used?

up = give back book; **down** = take book



implementing mutexes with semaphores

```
struct Mutex {  
    Semaphore s; /* with initial value 1 */  
    /* value = 1 --> mutex is free */  
    /* value = 0 --> mutex is busy */  
}
```

```
MutexLock(Mutex *m) {  
    m->s.down();  
}
```

```
MutexUnlock(Mutex *m) {  
    m->s.up();  
}
```

implementing join with semaphores

```
struct Thread {  
    ...  
    Semaphore finish_semaphore; /* with initial value 0 */  
    /* value = 0: either thread not finished OR already joined */  
    /* value = 1: thread finished AND not joined */  
};  
thread_join(Thread *t) {  
    t->finish_semaphore.down();  
}  
  
/* assume called when thread finishes */  
thread_exit(Thread *t) {  
    t->finish_semaphore.up();  
    /* tricky part: deallocating struct Thread safely? */  
}
```

POSIX semaphores

```
#include <semaphore.h>
...
sem_t my_semaphore;
int process_shared = /* 1 if sharing between processes */;
sem_init(&my_semaphore, process_shared, initial_value);
...
sem_wait(&my_semaphore); /* down */
sem_post(&my_semaphore); /* up */
...
sem_destroy(&my_semaphore);
```

semaphore exercise

```
int value;  sem_t empty, ready;  // with some initial values
void PutValue(int argument) {
    sem_wait(&empty);
    value = argument;
    sem_post(&ready);
}
int GetValue() {
    int result;

    -----
    result = value;

    -----
    return result;
}
```

GetValue() waits for PutValue() to happen, retrieves value, then allows next PutValue().
PutValue() waits for prior GetValue(), places value, then allows next GetValue().

What goes in the blanks?

- a. sem_post(&empty) / sem_wait(&ready) d. sem_post(&ready) / sem_post(&empty)
- b. sem_wait(&ready) / sem_post(&empty) e. sem_wait(&empty) / sem_post(&ready)
- c. sem_post(&ready) / sem_wait(&empty) f. something else

semaphore exercise solution

```
int value;
sem_t empty, ready;
void PutValue(int argument) {
    sem_wait(&empty);
    value = argument;
    sem_post(&ready);
}
int GetValue() {
    int result;
    sem_wait(&ready);
    result = value;
    sem_post(&empty);
    return result;
}
```


semaphore intuition

What do you need to wait for?

- critical section to be finished

- queue to be non-empty

- array to have space for new items

what can you count that will be 0 when you need to wait?

- # of threads that can start critical section now

- # of threads that can join another thread without waiting

- # of items in queue

- # of empty spaces in array

use up/down operations to maintain count

producer/consumer constraints

consumer waits for producer(s) if buffer is empty

producer waits for consumer(s) if buffer is full

any thread waits while a thread is manipulating the buffer

one semaphore per constraint:

```
sem_t full_slots;    // consumer waits if empty
sem_t empty_slots;   // producer waits if full
sem_t mutex;         // either waits if anyone changing buffer
FixedSizedQueue buffer;
```

producer/consumer pseudocode

```
1  sem_init(&full_slots, ..., 0);
2  sem_init(&empty_slots, ..., BUFFER_CAPACITY);
3  sem_init(&mutex, ..., 1 /* # thread that can use buffer at once */);
4  buffer.set_size(BUFFER_CAPACITY);
5  ...
6  Produce(item) {
7      sem_wait(&empty_slots);
8      sem_wait(&mutex);
9      buffer.enqueue(item);
10     sem_post(&mutex);
11     // tell consumers there is more data
12     sem_post(&full_slots);
13 }
14
15 Consume() {
16     // wait until queued item, reserve it
17     sem_wait(&full_slots);
18     sem_wait(&mutex);
19     item = buffer.dequeue();
20     sem_post(&mutex);
21     // let producer reuse item slot
22     sem_post(&empty_slots);
23     return item;
24 }
25
```


producer/consumer pseudocode — semaphores

```
1  sem_init(&full_slots, ..., 0);
2  sem_init(&empty_slots, ..., BUFFER_CAPACITY);
3  sem_init(&mutex, ..., 1 /* # thread that can use buffer at once */);
4  buffer.set_size(BUFFER_CAPACITY);
5  ...
6  Produce(item) {
7      sem_wait(&empty_slots);
8      sem_wait(&mutex);
9      buffer.enqueue(item);
10     sem_post(&mutex);
11     // tell consumers there is more data
12     sem_post(&full_slots);
13 }
14
15 Consume() {
16     // wait until queued item, reserve it
17     sem_wait(&full_slots);
18     sem_wait(&mutex);
19     item = buffer.dequeue();
20     sem_post(&mutex);
21     // let producer reuse item slot
22     sem_post(&empty_slots);
23     return item;
24 }
25
```

$\text{full_slots} \leq \# \text{ items on queue}$
 $\text{empty_slots} \leq \# \text{ free slots on queue}$

producer/consumer pseudocode — exercise 1

```
1  sem_init(&full_slots, ..., 0);
2  sem_init(&empty_slots, ..., BUFFER_CAPACITY);
3  sem_init(&mutex, ..., 1 /* # thread that can use buffer at once */);
4  buffer.set_size(BUFFER_CAPACITY);
5  ...
6  Produce(item) {
7      sem_wait(&empty_slots);
8      sem_wait(&mutex);
9      buffer.enqueue(item);
10     sem_post(&mutex);
11     // tell consumers there is more data
12     sem_post(&full_slots);
13 }
14
15 Consume() {
16     // wait until queued item, reserve it
17     sem_wait(&full_slots);
18     sem_wait(&mutex);
19     item = buffer.dequeue();
20     sem_post(&mutex);
21     // let producer reuse item slot
22     sem_post(&empty_slots);
23     return item;
24 }
25
```

exercise: when is `full_slots` value + `empty_slots` value $\neq$ queue size?

producer/consumer pseudocode exercise 2

```
1  sem_init(&full_slots, ..., 0);
2  sem_init(&empty_slots, ..., BUFFER_CAPACITY);
3  sem_init(&mutex, ..., 1 /* # thread that can
4  buffer.set_size(BUFFER_CAPACITY);
5  ...
6  Produce(item) {
7      sem_wait(&empty_slots);
8      sem_wait(&mutex);
9      buffer.enqueue(item);
10     sem_post(&mutex);
11     // tell consumers there is more data
12     sem_post(&full_slots);
13 }
14
15 Consume() {
16     // wait until queued item, reserve it
17     sem_wait(&full_slots);
18     sem_wait(&mutex);
19     item = buffer.dequeue();
20     sem_post(&mutex);
21     // let producer reuse item slot
22     sem_post(&empty_slots);
23     return item;
24 }
25
```

can we do

```
sem_wait(&mutex);
sem_wait(&empty_slots);
```

instead in Produce()?

producer/consumer pseudocode exercise 2

```
1  sem_init(&full_slots, ..., 0);
2  sem_init(&empty_slots, ..., BUFFER_CAPACITY);
3  sem_init(&mutex, ..., 1 /* # thread that can
4  buffer.set_size(BUFFER_CAPACITY);
5  ...
6  Produce(item) {
7      sem_wait(&empty_slots);
8      sem_wait(&mutex);
9      buffer.enqueue(item);
10     sem_post(&mutex);
11     // tell consumers there is more data
12     sem_post(&full_slots);
13 }
14
15 Consume() {
16     // wait until queued item, reserve it
17     sem_wait(&full_slots);
18     sem_wait(&mutex);
19     item = buffer.dequeue();
20     sem_post(&mutex);
21     // let producer reuse item slot
22     sem_post(&empty_slots);
23     return item;
24 }
25
```

can we do

```
sem_wait(&mutex);
sem_wait(&empty_slots);
```

instead in Produce()?

No Consumer waits on
sem_wait(&mutex)
so can't sem_wait(&empty_slots)
(deadlock)

producer/consumer: cannot reorder mutex/empty

```
ProducerReordered() {  
    // BROKEN: WRONG ORDER  
    sem_wait(&mutex);  
    sem_wait(&empty_slots);  
  
    ...  
  
    sem_post(&mutex);  
}
```

```
Consumer() {  
    sem_wait(&full_slots);  
  
    // can't finish until  
    // Producer's sem_post(&mutex):  
    sem_wait(&mutex);  
  
    ...  
  
    // so this is not reached  
    sem_post(&full_slots);  
}
```


producer/consumer pseudocode — exercise 3

```
1  sem_init(&full_slots, ..., 0);
2  sem_init(&empty_slots, ..., BUFFER_CAPACITY);
3  sem_init(&mutex, ..., 1 /* # thread that can use buffer at once */);
4  buffer.set_size(BUFFER_CAPACITY);
5  ...
6  Produce(item) {
7      sem_wait(&empty_slots);
8      sem_wait(&mutex);
9      buffer.enqueue(item);
10     sem_post(&mutex);
11     // tell consumers there is more data
12     sem_post(&full_slots);
13 }
14
15 Consume() {
16     // wait until queued item, reserve it
17     sem_wait(&full_slots);
18     sem_wait(&mutex);
19     item = buffer.dequeue();
20     sem_post(&mutex);
21     // let producer reuse item slot
22     sem_post(&empty_slots);
23     return item;
24 }
25
```

can we do

```
sem_post(&full_slots);
sem_post(&mutex);
```

instead in Produce()?

producer/consumer pseudocode — exercise 3

```
1  sem_init(&full_slots, ..., 0);
2  sem_init(&empty_slots, ..., BUFFER_CAPACITY);
3  sem_init(&mutex, ..., 1 /* # thread that can use buffer at once */);
4  buffer.set_size(BUFFER_CAPACITY);
5  ...
6  Produce(item) {
7      sem_wait(&empty_slots);
8      sem_wait(&mutex);
9      buffer.enqueue(item);
10     sem_post(&mutex);
11     // tell consumers there is more data
12     sem_post(&full_slots);
13 }
14
15 Consume() {
16     // wait until queued item, reserve it
17     sem_wait(&full_slots);
18     sem_wait(&mutex);
19     item = buffer.dequeue();
20     sem_post(&mutex);
21     // let producer reuse item slot
22     sem_post(&empty_slots);
23     return item;
24 }
25
```

can we do

```
sem_post(&full_slots);
sem_post(&mutex);
```

instead in Produce()?

Yes

producer/consumer summary

producer: wait (down) empty_slots, post (up) full_slots

consumer: wait (down) full_slots, post (up) empty_slots

two producers or consumers?

still works!

transactions

transaction: set of operations that occurs atomically

idea: something higher-level handles locking, etc.:

```
BeginTransaction();  
int FromOldBalance = GetBalance(FromAccount);  
int ToOldBalance = GetBalance(ToAccount);  
SetBalance(FromAccount, FromOldBalance - 100);  
SetBalance(ToAccount, ToOldBalance + 100);  
EndTransaction();
```

idea: library/database/etc. makes “transaction” happens all at once

consistency / durability

“happens all at once” = could mean:

locking to make sure no other operations interfere (consistency)

making sure on crash, no partial transaction seen (durability)

(some systems provide both, some provide only one)

we'll just talk about implementing consistency

transaction implementation ideas

(for consistency)

simple idea: do one transaction at a time

more efficient(?) idea:

- get list of operations

- lock everything operations use (in consistent order to prevent deadlock)

- do operations on list

- unlock everything

(also other strategies with different flexibility/efficiency)

read-copy-update (high-level overview)

idea: read-mostly data structure

when reading:

- read normally *via shared pointer*

when writing:

- make a copy

- atomically update the *shared pointer*

- delete the old version *eventually*

tricky part: when is it safe to delete old version

- implementation: *scheduler integration*

RCU operations

read lock — record: “I am reading now”

read unlock — record: “I am done reading now”

publish — atomically update pointer

after publish: wait until

- all threads currently running have context switched

- ... and none of them set the “I am reading now” bit

Backup slides

backup slides

a simple race

thread_A:

```
movl $1, x    /* x ← 1 */
movl y, %eax  /* return y */
ret
```

thread_B:

```
movl $1, y    /* y ← 1 */
movl x, %eax  /* return x */
ret
```

```
x = y = 0;
pthread_create(&A, NULL, thread_A, NULL);
pthread_create(&B, NULL, thread_B, NULL);
pthread_join(A, &A_result); pthread_join(B, &B_result);
printf("A:%d B:%d\n", (int) A_result, (int) B_result);
```

if loads/stores atomic, then possible results:

A:1 B:1 — both moves into x and y, then both moves into eax execute

A:0 B:1 — thread A executes before thread B

A:1 B:0 — thread B executes before thread A

a simple race: results

```
thread_A:
    movl $1, x    /* x <- 1 */
    movl y, %eax  /* return y */
    ret
```

```
thread_B:
    movl $1, y    /* y <- 1 */
    movl x, %eax  /* return x */
    ret
```

```
x = y = 0;
pthread_create(&A, NULL, thread_A, NULL);
pthread_create(&B, NULL, thread_B, NULL);
pthread_join(A, &A_result); pthread_join(B, &B_result);
printf("A:%d B:%d\n", (int) A_result, (int) B_result);
```

my desktop, 100M trials:

frequency	result	
99 823 739	A:0 B:1	(‘A executes before B’)
171 161	A:1 B:0	(‘B executes before A’)
4 706	A:1 B:1	(‘execute moves into x+y first’)
394	A:0 B:0	???

a simple race: results

```
thread_A:
    movl $1, x    /* x <- 1 */
    movl y, %eax  /* return y */
    ret
```

```
thread_B:
    movl $1, y    /* y <- 1 */
    movl x, %eax  /* return x */
    ret
```

```
x = y = 0;
pthread_create(&A, NULL, thread_A, NULL);
pthread_create(&B, NULL, thread_B, NULL);
pthread_join(A, &A_result); pthread_join(B, &B_result);
printf("A:%d B:%d\n", (int) A_result, (int) B_result);
```

my desktop, 100M trials:

frequency	result	
99 823 739	A:0 B:1	(‘A executes before B’)
171 161	A:1 B:0	(‘B executes before A’)
4 706	A:1 B:1	(‘execute moves into x+y first’)
394	A:0 B:0	???

why reorder here?

thread_A:

```
movl $1, x    /* x ← 1 */  
movl y, %eax  /* return y */  
ret
```

thread_B:

```
movl $1, y    /* y ← 1 */  
movl x, %eax  /* return x */  
ret
```

thread A: faster to load y right now!

... rather than wait for write of x to finish

why load/store reordering?

fast processor designs can execute instructions out of order

goal: do something instead of waiting for slow memory accesses, etc.

more on this later in the semester

GCC: preventing reordering example (1)

```
void WaitForReady() {  
    int one = 1;  
    __atomic_store(&is_waiting, &one, __ATOMIC_SEQ_CST);  
    do {  
    } while (!__atomic_load_n(&ready, __ATOMIC_SEQ_CST));  
    ...  
}
```

```
WaitForReady:  
    movl $1, is_waiting  
    mfence  
.L2:  
    movl ready, %eax  
    testl %eax, %eax  
    jz .L2  
    ...
```


GCC: preventing reordering example (2)

```
void WaitForReady() {
    is_waiting = 1;
    do {
        __atomic_thread_fence(__ATOMIC_SEQ_CST);
    } while (!ready);
    ...
}
```

```
WaitForReady:
    movl $1, is_waiting // is_waiting <- 1
.L3:
    mfence // make sure store is visible to other cores before loading
           // on x86: not needed on second+ iteration of loop
    cmpl $0, ready // if (ready == 0) repeat fence
    jz .L3
    ...
```

C++: preventing reordering

to help *implementing things like pthread_mutex_lock*

C++ 2011 standard: *atomic* header, *std::atomic* class

prevent CPU reordering *and* prevent compiler reordering

also provide other tools for implementing locks (more later)

could also hand-write assembly code

 compiler can't know what assembly code is doing

C++: preventing reordering example

```
#include <atomic>
void WaitForReady() {
    is_waiting = 1;
    do {
        std::atomic_thread_fence(std::memory_order_seq_cst);
    } while (!ready);
}
```

```
WaitForReady:
    movl $1, is_waiting // is_waiting <- 1
.L2:
    mfence // make sure store visible on/from other cores
    cmpl $0, ready // if (ready == 0) repeat fence
    jz .L2
    ...
```


C++ atomics: no reordering

```
std::atomic<int> is_waiting, ready;
void WaitForReady() {
    is_waiting.store(1);
    do {
    } while (ready.load());
}
```

```
WaitForReady:
    movl $1, is_waiting
    mfence
.L2:
    movl ready, %eax
    testl %eax, %eax
    jz .L2
    ...
```

GCC: built-in atomic functions

used to implement `std::atomic`, etc.

predate `std::atomic`

builtin functions starting with `__sync` and `__atomic`

aside: some x86 reordering rules

each core sees its own loads/stores in order

(if a core stores something, it can always load it back)

stores *from other cores* appear in a consistent order

(but a core might observe its own stores too early)

causality:

if a core reads $X=a$ and (after reading $X=a$) writes $Y=b$,

then a core that reads $Y=b$ cannot later read X =older value than a

how do you do anything with this?

difficult to reason about what modern CPU's reordering rules do
typically: don't depend on details, instead:

special instructions with stronger (and simpler) ordering rules
often same instructions that help with implementing locks in other ways

special instructions that restrict ordering of instructions around them ("fences")
loads/stores can't cross the fence

mfence

x86 instruction mfence

make sure all loads/stores in progress finish

... and make sure no loads/stores were started early

fairly expensive

Intel 'Skylake': order 33 cycles + time waiting for pending stores/loads

aside: this instruction did not exist in the original x86

so xv6 uses something older that's equivalent

backup slides

Java synchronized primitive

```
Object MilkLock = new Object();
```

```
/* lock implicity acquired/released on  
   entering/leaving this block */  
synchronized (MilkLock) {  
    if (no milk) {  
        buy milk  
    }  
}
```

C++11 mutexes

```
#include <mutex>

std::mutex MilkLock;

{
    std::lock_guard nameDoesNotMatter(MilkLock);
    /* nameDoesNotMatter's constructor acquires lock */
    if (no milk) {
        buy milk
    }
    /* nameDoesNotMatter's destructor called automatically
       and releases lock
    */
}
```


are locks enough?

do we need more than locks?

example 1: pipes?

pipes: one thread reads while other writes

want write to complete immediately if buffer space

want read operation to *wait* for write operation

not functionality provided by mutexes/barriers

pthread mutexes: addt'l features

mutex attributes (`pthread_mutexattr_t`) allow:
(reference: `man pthread.h`)

error-checking mutexes

- locking mutex twice in same thread?

- unlocking already unlocked mutex?

- ...

mutexes shared between processes

- otherwise: must be only threads of same process

- (unanswered question: where to store mutex?)

- ...

recall: pthread mutex

```
#include <pthread.h>

pthread_mutex_t some_lock;
pthread_mutex_init(&some_lock, NULL);
// or: pthread_mutex_t some_lock = PTHREAD_MUTEX_INITIALIZER;
...
pthread_mutex_lock(&some_lock);
...
pthread_mutex_unlock(&some_lock);
pthread_mutex_destroy(&some_lock);
```

C++ containers and locking

can you use a vector from multiple threads?

... question: how is it implemented?

- dynamically allocated array
- reallocated on size changes

can access from multiple threads ... *as long as not append/erase/etc.?*

assuming it's implemented like we expect...

- but can we really depend on that?

- e.g. could shrink internal array after a while with no expansion save memory?

C++ standard rules for containers

multiple threads can *read anything at the same time*

can only read element *if no other thread is modifying it*

can safely *add/remove elements if no other threads* are accessing container
(sometimes can safely add/remove in extra cases)

exception: vectors of bools — can't safely read and write at same time
might be implemented by putting multiple bools in one int

why spurious wakeups?

makes implementing condition variables simpler

can be hard to avoid loop in more complicated scenarios

e.g. `signal()` saying okay to remove item from queue

what if another thread sneaks in and does it first?

maybe `signal()` could be redesigned to prevent this somehow?

... but that's harder to implement

BROKEN: producer/consumer signal

exercise: example why signal here is BROKEN? hint: two consume()+two produce()

```
pthread_mutex_t lock; pthread_cond_t data_ready; UnboundedQueue buffer;
Produce(item) {
    pthread_mutex_lock(&lock);
    buffer.enqueue(item);
    /* GOOD CODE: pthread_cond_signal(&data_ready); */
    /* BAD CODE: */ if (buffer.size() == 1) pthread_cond_signal(&item);
    pthread_mutex_unlock(&lock);
}
Consume() {
    pthread_mutex_lock(&lock);
    while (buffer.empty()) {X\\tikzmark{empty}X
        pthread_cond_wait(&data_ready, &lock);
    }X\\tikzmark{after loop}X
    item = buffer.dequeue();
    pthread_mutex_unlock(&lock);
    return item;
}
```


BROKEN: producer/consumer signal

exercise: example why signal here is BROKEN? hint: two consume()+two produce()

```
pthread_mutex_t lock; pthread_cond_t data_ready; UnboundedQueue buffer;
Produce(item) {
    pthread_mutex_lock(&lock);
    buffer.enqueue(item);
    /* GOOD CODE: pthread_cond_signal(&data_ready); */
    /* BAD CODE: */ if (buffer.size() == 1) pthread_cond_signal(&item);
    pthread_mutex_unlock(&lock);
}
Consume() {
    pthread_mutex_lock(&lock);
    while (buffer.empty()) {X\\tikzmark{empty}X
        pthread_cond_wait(&data_ready, &lock);
    }X\\tikzmark{after loop}X
    item = buffer.dequeue();
    pthread_mutex_unlock(&lock);
    return item;
}
```

bad case (setup)

thread 0

1

2

3

\hline{} Consume():

lock

empty? wait on cv

Consume():

lock

empty? wait on cv

Produce():

lock

Produce():

againframe(PCSignalBadSetup)

bad case

thread 0

\hline{} Consume():

lock

empty? wait on cv

wait for lock

1

Consume():

lock

empty? wait on cv

2

Produce():

lock

enqueue

size = 1? signal

unlock

3

Produce():
wait for lock

gets lock
enqueue

size $\neq$ 1: don't signal

againframe(PCSignalBad)

monitor exercise: ConsumeTwo

suppose we want producer/consumer, but...

but change Consume() to ConsumeTwo() which returns a *pair of values*
and don't want two calls to ConsumeTwo() to wait...
with each getting one item

what should we change below?

```
pthread_mutex_t lock;  
pthread_cond_t data_ready;  
UnboundedQueue buffer;  
  
Produce(item) {  
    pthread_mutex_lock(&lock);  
    buffer.enqueue(item);  
    pthread_cond_signal(&data_ready);  
    pthread_mutex_unlock(&lock);  
}
```

```
Consume() {  
    pthread_mutex_lock(&lock);  
    while (buffer.empty()) {  
        pthread_cond_wait(&data_ready, &lock);  
    }  
    item = buffer.dequeue();  
    pthread_mutex_unlock(&lock);  
    return item;  
}
```

monitor exercise: solution (1)

(one of many possible solutions)

Assuming ConsumeTwo *replaces* Consume:

```
Produce() {  
    pthread_mutex_lock(&lock);  
    buffer.enqueue(item);  
    if (buffer.size() > 1) { pthread_cond_signal(&data_ready); }  
    pthread_mutex_unlock(&lock);  
}  
ConsumeTwo() {  
    pthread_mutex_lock(&lock);  
    while (buffer.size() < 2) { pthread_cond_wait(&data_ready, &lock); }  
    item1 = buffer.dequeue(); item2 = buffer.dequeue();  
    pthread_mutex_unlock(&lock);  
    return Combine(item1, item2);  
}
```

monitor exercise: solution (2)

(one of many possible solutions)

Assuming ConsumeTwo is *in addition to* Consume (using two CVs):

```
Produce() {
    pthread_mutex_lock(&lock);
    buffer.enqueue(item);
    pthread_cond_signal(&one_ready);
    if (buffer.size() > 1) { pthread_cond_signal(&two_ready); }
    pthread_mutex_unlock(&lock);
}

Consume() {
    pthread_mutex_lock(&lock);
    while (buffer.size() < 1) { pthread_cond_wait(&one_ready, &lock); }
    item = buffer.dequeue();
    pthread_mutex_unlock(&lock);
    return item;
}

ConsumeTwo() {
    pthread_mutex_lock(&lock);
    while (buffer.size() < 2) { pthread_cond_wait(&two_ready, &lock); }
    item1 = buffer.dequeue(); item2 = buffer.dequeue();
    pthread_mutex_unlock(&lock);
    return Combine(item1, item2);
}
```


monitor exercise: slower solution

(one of many possible solutions)

Assuming ConsumeTwo is *in addition to* Consume (using one CV):

```
Produce() {
    pthread_mutex_lock(&lock);
    buffer.enqueue(item);
    // broadcast and not signal, b/c we might wakeup only ConsumeTwo() otherwise
    pthread_cond_broadcast(&data_ready);
    pthread_mutex_unlock(&lock);
}

Consume() {
    pthread_mutex_lock(&lock);
    while (buffer.size() < 1) { pthread_cond_wait(&data_ready, &lock); }
    item = buffer.dequeue();
    pthread_mutex_unlock(&lock);
    return item;
}

ConsumeTwo() {
    pthread_mutex_lock(&lock);
    while (buffer.size() < 2) { pthread_cond_wait(&data_ready, &lock); }
    item1 = buffer.dequeue(); item2 = buffer.dequeue();
    pthread_mutex_unlock(&lock);
    return Combine(item1, item2);
}
```

exercise: WaitForBoth

exercise: wait for both finished

```
pthread_mutex_t lock; pthread_cond_t cv;
bool FirstFinished = false; bool SecondFinished = false;

void FinishFirst() {
    pthread_mutex_lock(&lock);
    FirstFinished = true;
    _____ // (1)
    pthread_mutex_unlock(&lock);
}

void FinishSecond() {
    pthread_mutex_lock(&lock);
    SecondFinished = true;
    _____ // (1)
    pthread_mutex_unlock(&lock);
}

void WaitForBothFinished() {
    pthread_mutex_lock(&lock);
    ___ ( _____ ) { // (2)
        pthread_cond_wait(&lock, &cv);
    }
    pthread_mutex_unlock(&lock);
}
```

Fill in the blanks.

monitor exercise: ordering

suppose we want producer/consumer, but...

but want to ensure first call to Consume() *always* returns first
(no matter what ordering cond_signal/cond_broadcast use)

```
pthread_mutex_t lock;  
pthread_cond_t data_ready;  
UnboundedQueue buffer;
```

```
Produce(item) {  
    pthread_mutex_lock(&lock);  
    buffer.enqueue(item);  
    pthread_cond_signal(&data_ready);  
    pthread_mutex_unlock(&lock);  
}
```

```
Consume() {  
    pthread_mutex_lock(&lock);  
    while (buffer.empty()) {  
        pthread_cond_wait(&data_ready, &lock);  
    }  
    item = buffer.dequeue();  
    pthread_mutex_unlock(&lock);  
    return item;  
}
```

monitor ordering exercise: solution

(one of many possible solutions)

... { .columns }

```
struct Waiter {  
    pthread_cond_t cv;  
    bool done;  
    T item;  
}  
Queue<Waiter*> waiters;
```

```
Produce(item) {  
    pthread_mutex_lock(&lock);  
    if (!waiters.empty()) {  
        Waiter *waiter = waiters.dequeue();  
        waiter->done = true;  
        waiter->item = item;  
        cond_signal(&waiter->cv);  
        ++num_pending;  
    } else {  
        buffer.enqueue(item);  
    }  
    pthread_mutex_unlock(&lock);  
}
```

...

```
Consume() {  
    pthread_mutex_lock(&lock);  
    if (buffer.empty()) {  
        Waiter waiter;  
        cond_init(&waiter.cv);  
        waiter.done = false;  
        waiters.enqueue(&waiter);  
        while (!waiter.done)  
            cond_wait(&waiter.cv, &lock);  
        item = waiter.item;  
    } else {  
        item = buffer.dequeue();  
    }  
    pthread_mutex_unlock(&lock);  
    return item;  
}
```


bounded buffer producer/consumer

```
pthread_mutex_t lock;  
pthread_cond_t data_ready; pthread_cond_t @4space_ready4@;  
BoundedQueue buffer;
```

```
Produce(item) {  
    pthread_mutex_lock(&lock);  
    while (buffer.full()) { pthread_cond_wait(@4&space_ready4@, &lock); }  
    buffer.enqueue(item);  
    pthread_cond_signal(&data_ready);  
    pthread_mutex_unlock(&lock);  
}
```

```
Consume() {  
    pthread_mutex_lock(&lock);  
    while (buffer.empty()) {  
        pthread_cond_wait(&data_ready, &lock);  
    }  
    item = buffer.dequeue();  
    [[pthread_cond_signal(@4&space_ready4@);] {.fragment fragment-index=2 .custom .myem-only}][.fragment  
fragment-index=3 .custom .myem-only}X\\tikzmark{signal}X
```

bounded buffer producer/consumer

```
pthread_mutex_t lock;  
pthread_cond_t data_ready; pthread_cond_t @4space_ready4@;  
BoundedQueue buffer;
```

```
Produce(item) {  
    pthread_mutex_lock(&lock);  
    while (buffer.full()) { pthread_cond_wait(@4&space_ready4@, &lock); }  
    buffer.enqueue(item);  
    pthread_cond_signal(&data_ready);  
    pthread_mutex_unlock(&lock);  
}
```

```
Consume() {  
    pthread_mutex_lock(&lock);  
    while (buffer.empty()) {  
        pthread_cond_wait(&data_ready, &lock);  
    }  
    item = buffer.dequeue();  
    [[pthread_cond_signal(@4&space_ready4@);] {.fragment fragment-index=2 .custom .myem-only}][.fragment  
fragment-index=3 .custom .myem-only}X\\tikzmark{signal}X
```

bounded buffer producer/consumer

```
pthread_mutex_t lock;  
pthread_cond_t data_ready; pthread_cond_t @4space_ready4@;  
BoundedQueue buffer;
```

```
Produce(item) {  
    pthread_mutex_lock(&lock);  
    while (buffer.full()) { pthread_cond_wait(@4&space_ready4@, &lock); }  
    buffer.enqueue(item);  
    pthread_cond_signal(&data_ready);  
    pthread_mutex_unlock(&lock);  
}
```

```
Consume() {  
    pthread_mutex_lock(&lock);  
    while (buffer.empty()) {  
        pthread_cond_wait(&data_ready, &lock);  
    }  
    item = buffer.dequeue();  
    [[pthread_cond_signal(@4&space_ready4@);] {.fragment fragment-index=2 .custom .myem-only}][.fragment  
fragment-index=3 .custom .myem-only}X\\tikzmark{signal}X
```


potential fixes

unconditionally signal

- each consume allows one produce to go

- rely on condition variable knowing if no one is waiting

broadcast if buffer changed from full to not-full

- every thread waiting because it was full could go buffer it becomes full again

explicitly count number of waiting producers — buffer not full and waiter

how could I have avoided this?

question: who might be waiting when condition changes
almost always multiple threads!

if not broadcasting, *explain why each waiting thread gets to go*

my implicit non-explanation: queue will be full again first
not actually true: can keep consuming before producers go

alternate view: consuming causes what threads to go?
not just when the buffer was full
since if I empty the buffer by consuming...

rwlocks with monitors (attempt 1)

```
mutex_t lock;
```

rwlocks with monitors (attempt 1)

```
mutex_t lock;
```

rwlocks with monitors (attempt 1)

```
mutex_t lock;
```

```
unsigned int readers, writers;
```


rwlocks with monitors (attempt 1)

```
mutex_t lock;  
unsigned int readers, writers;  
/* condition, signal when writers becomes 0 */  
cond_t ok_to_read_cv;  
/* condition, signal when readers + writers becomes 0 */  
cond_t ok_to_write_cv;
```

rwlocks with monitors (attempt 1)

```
mutex_t lock;

unsigned int readers, writers;

/* condition, signal when writers becomes 0 */
cond_t ok_to_read_cv;
/* condition, signal when readers + writers becomes 0 */
cond_t ok_to_write_cv;
```

```
ReadLock() {
    mutex_lock(&lock);
    while (writers != 0) {
        cond_wait(&ok_to_read_cv, &lock);
    }
    ++readers;
    mutex_unlock(&lock);
}
```

```
ReadUnlock() {
    mutex_lock(&lock);
    --readers;
    if (readers == 0) {
        cond_signal(&ok_to_write_cv);
    }
    mutex_unlock(&lock);
}
```

```
WriteLock() {
    mutex_lock(&lock);
    while (readers + writers != 0) {
        cond_wait(&ok_to_write_cv);
    }
    ++writers;
    mutex_unlock(&lock);
}

WriteUnlock() {
    mutex_lock(&lock);
    --writers;
    [[cond_signal(&ok_to_write_cv);] {.fragment
fragment-index=5 .custom .myem-only}]{.fragment
fragment-index=6 .custom .myem-only}
    [[cond_broadcast(&ok_to_read_cv);] {.fragment
fragment-index=4 .custom .myem-only}]{.fragment
fragment-index=6 .custom .myem-only}
    mutex_unlock(&lock);
}
```

rwlocks with monitors (attempt 1)

```
mutex_t lock;

unsigned int readers, writers;

/* condition, signal when writers becomes 0 */
cond_t ok_to_read_cv;
/* condition, signal when readers + writers becomes 0 */
cond_t ok_to_write_cv;
```

```
ReadLock() {
    mutex_lock(&lock);
    while (writers != 0) {
        cond_wait(&ok_to_read_cv, &lock);
    }
    ++readers;
    mutex_unlock(&lock);
}
```

```
ReadUnlock() {
    mutex_lock(&lock);
    --readers;
    if (readers == 0) {
        cond_signal(&ok_to_write_cv);
    }
    mutex_unlock(&lock);
}
```

```
WriteLock() {
    mutex_lock(&lock);
    while (readers + writers != 0) {
        cond_wait(&ok_to_write_cv);
    }
    ++writers;
    mutex_unlock(&lock);
}

WriteUnlock() {
    mutex_lock(&lock);
    --writers;
    [[cond_signal(&ok_to_write_cv);] {.fragment
fragment-index=5 .custom .myem-only}]{.fragment
fragment-index=6 .custom .myem-only}
    [[cond_broadcast(&ok_to_read_cv);] {.fragment
fragment-index=4 .custom .myem-only}]{.fragment
fragment-index=6 .custom .myem-only}
    mutex_unlock(&lock);
}
```


rwlocks with monitors (attempt 1)

```
mutex_t lock;

unsigned int readers, writers;

/* condition, signal when writers becomes 0 */
cond_t ok_to_read_cv;
/* condition, signal when readers + writers becomes 0 */
cond_t ok_to_write_cv;
```

```
ReadLock() {
    mutex_lock(&lock);
    while (writers != 0) {
        cond_wait(&ok_to_read_cv, &lock);
    }
    ++readers;
    mutex_unlock(&lock);
}
```

```
ReadUnlock() {
    mutex_lock(&lock);
    --readers;
    if (readers == 0) {
        cond_signal(&ok_to_write_cv);
    }
    mutex_unlock(&lock);
}
```

```
WriteLock() {
    mutex_lock(&lock);
    while (readers + writers != 0) {
        cond_wait(&ok_to_write_cv);
    }
    ++writers;
    mutex_unlock(&lock);
}

WriteUnlock() {
    mutex_lock(&lock);
    --writers;
    [[cond_signal(&ok_to_write_cv);] {.fragment
fragment-index=5 .custom .myem-only}]{.fragment
fragment-index=6 .custom .myem-only}
    [[cond_broadcast(&ok_to_read_cv);] {.fragment
fragment-index=4 .custom .myem-only}]{.fragment
fragment-index=6 .custom .myem-only}
    mutex_unlock(&lock);
}
```

writer-priority (1)

```
mutex_t lock; cond_t ok_to_read_cv; cond_t ok_to_write_cv;
int readers = 0, writers = 0;
int waiting_writers = 0;
```

```
ReadLock() {
    mutex_lock(&lock);
    while (writers != 0
           || waiting_writers != 0) {
        cond_wait(&ok_to_read_cv, &lock);
    }
    ++readers;
    mutex_unlock(&lock);
}
```

```
ReadUnlock() {
    mutex_lock(&lock);
    --readers;
    if (readers == 0) {
        cond_signal(&ok_to_write_cv);
    }
    mutex_unlock(&lock);
}
```

```
WriteLock() {
    mutex_lock(&lock);
    ++waiting_writers;
    while (readers + writers != 0) {
        cond_wait(&ok_to_write_cv, &lock);
    }
    --waiting_writers;
    ++writers;
    mutex_unlock(&lock);
}
```

```
WriteUnlock() {
    mutex_lock(&lock);
    --writers;
    if (waiting_writers != 0) {
        cond_signal(&ok_to_write_cv);
    } else {
        cond_broadcast(&ok_to_read_cv);
    }
    mutex_unlock(&lock);
}
```

writer-priority (1)

```
mutex_t lock; cond_t ok_to_read_cv; cond_t ok_to_write_cv;
int readers = 0, writers = 0;
int waiting_writers = 0;
```

```
ReadLock() {
    mutex_lock(&lock);
    while (writers != 0
           || waiting_writers != 0) {
        cond_wait(&ok_to_read_cv, &lock);
    }
    ++readers;
    mutex_unlock(&lock);
}
```

```
ReadUnlock() {
    mutex_lock(&lock);
    --readers;
    if (readers == 0) {
        cond_signal(&ok_to_write_cv);
    }
    mutex_unlock(&lock);
}
```

```
WriteLock() {
    mutex_lock(&lock);
    ++waiting_writers;
    while (readers + writers != 0) {
        cond_wait(&ok_to_write_cv, &lock);
    }
    --waiting_writers;
    ++writers;
    mutex_unlock(&lock);
}
```

```
WriteUnlock() {
    mutex_lock(&lock);
    --writers;
    if (waiting_writers != 0) {
        cond_signal(&ok_to_write_cv);
    } else {
        cond_broadcast(&ok_to_read_cv);
    }
    mutex_unlock(&lock);
}
```

writer-priority (1)

```
mutex_t lock; cond_t ok_to_read_cv; cond_t ok_to_write_cv;
int readers = 0, writers = 0;
int waiting_writers = 0;
```

```
ReadLock() {
    mutex_lock(&lock);
    while (writers != 0
           || waiting_writers != 0) {
        cond_wait(&ok_to_read_cv, &lock);
    }
    ++readers;
    mutex_unlock(&lock);
}
```

```
ReadUnlock() {
    mutex_lock(&lock);
    --readers;
    if (readers == 0) {
        cond_signal(&ok_to_write_cv);
    }
    mutex_unlock(&lock);
}
```

```
WriteLock() {
    mutex_lock(&lock);
    ++waiting_writers;
    while (readers + writers != 0) {
        cond_wait(&ok_to_write_cv, &lock);
    }
    --waiting_writers;
    ++writers;
    mutex_unlock(&lock);
}
```

```
WriteUnlock() {
    mutex_lock(&lock);
    --writers;
    if (waiting_writers != 0) {
        cond_signal(&ok_to_write_cv);
    } else {
        cond_broadcast(&ok_to_read_cv);
    }
    mutex_unlock(&lock);
}
```

reader-priority (1)

...

```
int waiting_readers = 0;
```

```
ReadLock() {  
    mutex_lock(&lock);  
    ++waiting_readers;  
    while (writers != 0) {  
        cond_wait(&ok_to_read_cv, &lock);  
    }  
    --waiting_readers;  
    ++readers;  
    mutex_unlock(&lock);  
}
```

```
ReadUnlock() {  
    ...  
    if (waiting_readers == 0) {  
        cond_signal(&ok_to_write_cv);  
    }  
}
```

```
WriteLock() {  
    mutex_lock(&lock);  
    while (waiting_readers +  
           readers + writers != 0) {  
        cond_wait(&ok_to_write_cv);  
    }  
    ++writers;  
    mutex_unlock(&lock);  
}  
WriteUnlock() {  
    mutex_lock(&lock);  
    --writers;  
    if (readers == 0 && waiting_readers == 0) {  
        cond_signal(&ok_to_write_cv);  
    } else {  
        cond_broadcast(&ok_to_read_cv);  
    }  
    mutex_unlock(&lock);  
}
```


reader-priority (1)

...

```
int waiting_readers = 0;
```

```
ReadLock() {  
    mutex_lock(&lock);  
    ++waiting_readers;  
    while (writers != 0) {  
        cond_wait(&ok_to_read_cv, &lock);  
    }  
    --waiting_readers;  
    ++readers;  
    mutex_unlock(&lock);  
}
```

```
ReadUnlock() {  
    ...  
    if (waiting_readers == 0) {  
        cond_signal(&ok_to_write_cv);  
    }  
}
```

```
WriteLock() {  
    mutex_lock(&lock);  
    while (waiting_readers +  
           readers + writers != 0) {  
        cond_wait(&ok_to_write_cv);  
    }  
    ++writers;  
    mutex_unlock(&lock);  
}  
WriteUnlock() {  
    mutex_lock(&lock);  
    --writers;  
    if (readers == 0 && waiting_readers == 0) {  
        cond_signal(&ok_to_write_cv);  
    } else {  
        cond_broadcast(&ok_to_read_cv);  
    }  
    mutex_unlock(&lock);  
}
```

choosing orderings?

can use monitors to implement lots of lock policies

want X to go first/last — add extra variables

(number of waiters, even lists of items, etc.)

need way to write condition “you can go now”

e.g. writer-priority: readers can go if no writer waiting

Anderson-Dahlin and semaphores

Anderson/Dahlin complains about semaphores

“Our view is that programming with locks and condition variables is superior to programming with semaphores.”

argument 1: clearer to have *separate constructs* for
waiting for condition to become true, and
allowing only one thread to manipulate a thing at a time

argument 2: tricky to verify thread calls up exactly once for every down
alternatives allow one to be sloppier (in a sense)

monitors with semaphores: locks

```
sem_t semaphore; // initial value 1
```

```
Lock() {  
    sem_wait(&semaphore);  
}
```

```
Unlock() {  
    sem_post(&semaphore);  
}
```

monitors with semaphores: [broken] cvs

start with only wait/signal:

```
sem_t threads_to_wakeup; // initially 0
Wait(Lock lock) {
    lock.Unlock();
    sem_wait(&threads_to_wakeup);
    lock.Lock();
}
Signal() {
    sem_post(&threads_to_wakeup);
}
```

problem: *signal wakes up non-waiting threads (in the far future)*

monitors with semaphores: cvs (better)

start with only wait/signal:

```
sem_t private_lock; // initially 1
int num_waiters;
sem_t threads_to_wakeup; // initially 0

Wait(Lock lock) {
    sem_wait(&private_lock);
    ++num_waiters;
    sem_post(&private_lock);
    lock.Unlock();
    sem_wait(&threads_to_wakeup);
    lock.Lock();
}

Signal() {
    sem_wait(&private_lock);
    if (num_waiters > 0) {
        sem_post(&threads_to_wakeup);
        --num_waiters;
    }
    sem_post(&private_lock);
}
```

monitors with semaphores: broadcast

now allows broadcast:

```
sem_t private_lock; // initially 1
int num_waiters;
sem_t threads_to_wakeup; // initially 0
Wait(Lock lock) {
    sem_wait(&private_lock);
    ++num_waiters;
    sem_post(&private_lock);
    lock.Unlock();
    sem_wait(&threads_to_wakeup);
    lock.Lock();
}

Broadcast() {
    sem_wait(&private_lock);
    while (num_waiters > 0) {
        sem_post(&threads_to_wakeup);
        --num_waiters;
    }
    sem_post(&private_lock);
}
```

building semaphore with monitors

```
pthread_mutex_t lock;
```

lock to protect shared state

shared state: semaphore tracks a count

add cond var for each reason we wait

semaphore: wait for count to become positive (for down)

building semaphore with monitors

```
pthread_mutex_t lock;
```

lock to protect shared state

shared state: semaphore tracks a count

add cond var for each reason we wait

semaphore: wait for count to become positive (for down)

building semaphore with monitors

```
pthread_mutex_t lock;  
unsigned int count;
```

lock to protect shared state

shared state: semaphore tracks a count

add cond var for each reason we wait

semaphore: wait for count to become positive (for down)

building semaphore with monitors

```
pthread_mutex_t lock;  
unsigned int count;  
/* condition, broadcast when becomes count > 0 */  
pthread_cond_t count_is_positive_cv;
```

lock to protect shared state

shared state: semaphore tracks a count

add cond var for each reason we wait

semaphore: wait for count to become positive (for down)

building semaphore with monitors

```
pthread_mutex_t lock;

unsigned int count;

/* condition, broadcast when becomes count > 0 */
pthread_cond_t count_is_positive_cv;

void down() {
    pthread_mutex_lock(&lock);
    while (!(count > 0)) {
        pthread_cond_wait(
            &count_is_positive_cv,
            &lock);
    }
    count -= 1;
    pthread_mutex_unlock(&lock);
}
```

lock to protect shared state

shared state: semaphore tracks a count

add cond var for each reason we wait

semaphore: wait for count to become positive (for down)

building semaphore with monitors

```
pthread_mutex_t lock;

unsigned int count;

/* condition, broadcast when becomes count > 0 */
pthread_cond_t count_is_positive_cv;

void down() {
    pthread_mutex_lock(&lock);
    while (!(count > 0)) {
        pthread_cond_wait(
            &count_is_positive_cv,
            &lock);
    }
    count -= 1;
    pthread_mutex_unlock(&lock);
}

void up() {
    pthread_mutex_lock(&lock);
    count += 1;
    /* count must now be
       positive, and at most
       one thread can go per
       call to Up() */
    pthread_cond_signal(
        &count_is_positive_cv
    );
    pthread_mutex_unlock(&lock);
}
```

lock to protect shared state

shared state: semaphore tracks a count

add cond var for each reason we wait

semaphore: wait for count to become positive (for down)

binary semaphores

binary semaphores — semaphores that are *only zero or one*

as powerful as normal semaphores

exercise: simulate counting semaphores with binary semaphores
(more than one) and an integer

counting semaphores with binary semaphores (1)

\tiny{ via Hemmendinger, “Comments on ‘A correct and unrestrictive implementation of general semaphores’ ” (1989); Barz, “Implementing semaphores by binary semaphores” (1983)}

```
// assuming initialValue > 0
BinarySemaphore mutex(1);
int value = initialValue ;
BinarySemaphore gate(1 /* if initialValue >= 1 */);
    /* gate = # threads that can Down() now */
```

```
void Down() {
    gate.Down();
    // wait, if needed
    mutex.Down();
    value -= 1;
    if (value > 0) {
        gate.Up();
        // because next down should finish
        // now (but not marked to before)
    }
    mutex.Up();
}
```

```
void Up() {
    mutex.Down();
    value += 1;
    if (value == 1) {
        gate.Up();
        // because down should finish now
        // but could not before
    }
    mutex.Up();
}
```

gate intuition/pattern

pattern to allow one thread at a time:

```
sem_t gate; // 0 = closed; 1 = open
ReleasingThread() {
    ... // finish what the other thread is waiting for
    while (another thread is waiting and can go) {
        sem_post(&gate) // allow EXACTLY ONE thread
        ... // other bookkeeping
    }
    ...
}
WaitingThread() {
    ... // indicate that we're waiting
    sem_wait(&gate) // wait for gate to be open
    ... // indicate that we're not waiting
}
```


semaphores/CV

```
int num_waiting = 0;
bool finished = false;
sem_t mutex; // initially 1
sem_t gate;  // initially 0
void WaitForFinished() {
    sem_wait(&mutex);
    if (finished) {
        sem_post(&mutex);
    } else {
        num_waiting += 1;
        sem_post(&mutex);
        sem_wait(&gate);
    }
}

void Finish() {
    sem_wait(&mutex);
    finished = true;
    while (num_waiting > 0) {
        num_waiting -= 1;
    }
}
```

```
bool finished = false;
pthread_mutex_t mutex;
pthread_cond_t cv;

void WaitForFinished() {
    pthread_mutex_lock(&mutex);
    while (!finished) {
        pthread_cond_wait(&cv, &mutex);
    }
    pthread_mutex_unlock(&mutex);
}

void Finish() {
    pthread_mutex_lock(&mutex);
    finished = true;
    pthread_cond_broadcast(&cv);
    pthread_mutex_unlock(&mutex);
}
```

semaphores/CV

```
int num_waiting = 0;
bool finished = false;
sem_t mutex; // initially 1
sem_t gate;  // initially 0
void WaitForFinished() {
    sem_wait(&mutex);
    if (finished) {
        sem_post(&mutex);
    } else {
        num_waiting += 1;
        sem_post(&mutex);
        sem_wait(&gate);
    }
}

void Finish() {
    sem_wait(&mutex);
    finished = true;
    while (num_waiting > 0) {
        num_waiting -= 1;
    }
}
```

```
bool finished = false;
pthread_mutex_t mutex;
pthread_cond_t cv;

void WaitForFinished() {
    pthread_mutex_lock(&mutex);
    while (!finished) {
        pthread_cond_wait(&cv, &mutex);
    }
    pthread_mutex_unlock(&mutex);
}

void Finish() {
    pthread_mutex_lock(&mutex);
    finished = true;
    pthread_cond_broadcast(&cv);
    pthread_mutex_unlock(&mutex);
}
```

semaphores/CV

```
int num_waiting = 0;
bool finished = false;
sem_t mutex; // initially 1
sem_t gate;  // initially 0
void WaitForFinished() {
    sem_wait(&mutex);
    if (finished) {
        sem_post(&mutex);
    } else {
        num_waiting += 1;
        sem_post(&mutex);
        sem_wait(&gate);
    }
}

void Finish() {
    sem_wait(&mutex);
    finished = true;
    while (num_waiting > 0) {
        num_waiting -= 1;
    }
}
```

```
bool finished = false;
pthread_mutex_t mutex;
pthread_cond_t cv;

void WaitForFinished() {
    pthread_mutex_lock(&mutex);
    while (!finished) {
        pthread_cond_wait(&cv, &mutex);
    }
    pthread_mutex_unlock(&mutex);
}

void Finish() {
    pthread_mutex_lock(&mutex);
    finished = true;
    pthread_cond_broadcast(&cv);
    pthread_mutex_unlock(&mutex);
}
```


monitors with semaphores: chosen order

if we want to make sure threads woken up *in order*

```
ThreadSafeQueue<sem_t> waiters;
Wait(Lock lock) {
    sem_t private_semaphore;
    ... /* init semaphore
           with count 0 */
    waiters.Enqueue(&semaphore);
    lock.Unlock();
    sem_post(private_semaphore);
    lock.Lock();
}
```

```
Signal() {
    sem_t *next = waiters.DequeueOrNull();
    if (next != NULL) {
        sem_post(next);
    }
}
```

(but now implement queue with semaphores...)